# Beyond Bit-Width: SIMD Dequantization Overhead Creates a CPU/GPU Performance Divide in GGUF K-Quant LLM Inference

Kris D'Costa
*Halicioglu Data Science Institute*
*University of California, San Diego*
San Diego, CA, USA
kdcosta@ucsd.edu

***Abstract*—On CPU endpoints, the GPU heuristic—higher bit-width yields higher throughput—is wrong. We present a mechanistic, cross-platform study of GGUF K-quant inference on ARM (Pixel 6a) and x86 (Intel i5-1235U), finding three non-obvious results. (1) Throughput is non-monotonic: Q2_K leads at 7.49 tok/s (ARM) and 14.1 tok/s (x86), while Q6_K is *slowest* (3.53 and 6.8 tok/s)—a $2.1\times$ intra-format spread. The root cause is SIMD dequantization overhead: Q6_K's split-bit layout requires ≈26 NEON ops/block versus ≈10 for Q2_K (from static analysis of llama.cpp commit `1a29907`; see Table V). (2) Apple M4 Metal reverses the ordering entirely (Q4_K_S fastest, Q8_0 slowest), confirming the non-monotonicity is a CPU property; M4 CPU-only beats Metal for Q8_0 ($1.97\times$). (3) Q2_K exhibits a KV-cache context cliff ($-48\%$ ARM at ctx=512; $-50\%$ x86 at ctx=1300–1400), predicted within 5% by $L2_{\text{cache}}/1024$ on both platforms. Results validated on 1,200+ measurements across Llama 3.2 3B and Qwen 2.5 1.5B.**

***Index Terms*—LLM inference, quantization, GGUF, llama.cpp, ARM, x86, SIMD, KV-cache, mobile inference, edge AI, CPU/GPU divide, NEON, AVX2**

## I. Introduction

On-device LLM inference has converged on a de facto standard: GGUF K-quant formats [3] implemented in llama.cpp [2], running on CPU-only endpoints from mobile phones to edge servers. Practitioners choose among seven variants (Q2_K through Q8_0, spanning 2.6–8.5 bpw) guided by a single heuristic inherited from GPU literature: *higher bit-width yields higher throughput.* This paper shows that heuristic is wrong on every CPU backend tested, explains exactly why at the kernel instruction level, and derives a predictive model for when and how much it fails.

**Finding 1: Non-monotonic throughput, CPU-general.** On ARM (Pixel 6a, Cortex-X1 NEON) and x86 (Intel i5-1235U, AVX2), the six-bit variant Q6_K is the *slowest*: 3.53 tok/s on ARM and 6.8 tok/s on x86. The two-bit Q2_K is the *fastest*: 7.49 tok/s on ARM and 14.1 tok/s on x86—a $2.1\times$ intra-format spread in the wrong direction. The root cause is SIMD dequantization overhead. Q6_K stores weights split across two arrays (`ql[128]+qh[64]` per 256-weight superblock) and requires ≈26 NEON operations to reconstruct each block; Q2_K uses a single AND-mask and ≈10 operations (static analysis of `arm/quants.c`, commit `1a29907`; Table V). At the same memory bandwidth, dequantization cost dominates, inverting the expected ordering on both ISAs.

**Finding 2: CPU/GPU divide.** Apple M4 Metal reverses the ordering entirely: Q4_K_S leads at 19.9 tok/s, Q8_0 is slowest at 6.4 tok/s. On the same chip, M4 CPU-only inference outperforms Metal for Q8_0 (12.6 vs. 6.4 tok/s, $1.97\times$) and Q6_K (9.3 vs. 7.0 tok/s), because Metal's dedicated 4-bit dispatch paths eliminate the dequantization bottleneck that defines CPU behavior. The non-monotonic ordering is a CPU property, not a model property: it disappears the moment execution moves off general-purpose SIMD.

**Finding 3: KV-cache cliff with L2-predictable thresholds.** Q2_K throughput collapses $-48\%$ on ARM as context grows from 256 to 2048 tokens (onset at ctx=512), and $-50\%$ on x86 (onset at ctx=1300–1400). Both thresholds are predicted analytically: the per-layer combined K+V working set is $2048\times$ctx bytes [1], [12]; when this overflows L2 (512 KB on Cortex-X1, 1.25 MB on i5-1235U), attention computation becomes DRAM-latency-bound. The formula $\text{ctx}_{\text{cliff}} = L2/1024$ matches observation within 5% on both platforms. Q3_K_M is immune ($< \pm 11\%$) because its higher FFN compute cost masks the proportional impact of the attention overhead—a compute-masking effect, not a cache property.

**Why it matters.** These findings directly contradict the guidance embedded in llama.cpp documentation, Ollama model cards, and practitioner blogs, which uniformly recommend Q4–Q8 variants for quality and speed. On CPU, Q6_K is strictly dominated: slower than Q4_K_M on throughput *and* lower accuracy than Q4_K_S on BoolQ (65% vs. 74%). Q2_K is the fastest variant at short context but collapses $-48\%$ by ctx=2048 and fails near-randomly on HellaSwag MCQ tasks (19%, below the 25% random baseline). Correct deployment requires hardware-specific guidance, not bit-width heuristics.

**Contributions.** We present the first mechanistic, cross-platform characterization of GGUF K-quant inference on CPU, covering ARM NEON, x86 AVX2, and Apple Metal across 1,200+ measurements on Llama 3.2 3B and Qwen 2.5 1.5B [53]. We derive a kernel-level explanation of the throughput inversion (SIMD superblock dequantization cost), an analytical KV-cache overflow model that predicts cliff

TABLE I
COMPARISON WITH THE CLOSEST RELATED EMPIRICAL WORK. ✓= MEASURED AND EXPLAINED; ○ = OBSERVED BUT UNEXPLAINED; – = NOT ADDRESSED.

| Work | Platform | Non-mono. | KV cliff | Mech. | CPU/GPU |
|---|---|---|---|---|---|
| MELTing Point [18] | Mobile ARM/iOS | ○[a] | – | – | – |
| Xiao et al. [38] | Mobile SoC | ○ | – | – | ○ |
| MLC-LLM [7] | Mobile GPU | – | – | – | ○ |
| Na et al. [39] | Server CPU | – | – | ✓ | – |
| He et al. [45] | x86 CPU | – | – | ✓ | – |
| Gope et al. [43] | ARM CPU | – | – | ✓ | – |
| **This work** | ARM+x86+M4 | ✓ | ✓ | ✓ | ✓ |

[a] 3-bit < 4-bit observed [18]; coarser tiers, not K-quant.

thresholds from L2 size alone, and a CPU/GPU divide analysis showing the inversion is CPU-general and Metal-absent.

**Paper organization.** §2 surveys related work; §3 describes methodology; §4–5 present throughput and context-scaling results; §6 covers quality benchmarks; §7 gives the mechanistic analysis; §8 covers cross-platform validation; §9 gives deployment recommendations; §10 concludes.

## II. RELATED WORK

Table I positions this work against the most closely related empirical studies. We organize the discussion around five themes.

### A. Post-Training Quantization Methods

The dominant PTQ methods for GPU inference—GPTQ [4], AWQ [5], SmoothQuant [6], LLM.int8() [27], QuIP# [28], and SpQR [19]—all optimize for CUDA/Tensor Core execution paths where higher-bit representations generally improve throughput. These methods assume that dequantization cost is negligible (GPU fuses it with GEMM) and that weight layout is irrelevant to speed. Both assumptions break on CPU. Dettmers and Zettlemoyer [20] derived scaling laws showing 4-bit is Pareto-optimal for inference *quality*—consistent with our Q4_K_S finding— but these laws were derived on GPU and do not model CPU SIMD overheads.

Calibration-based quantization methods such as OBC [47] and SparseGPT [48] use second-order Hessian information to identify important weights, forming the theoretical basis for GGUF's imatrix calibration. Our evaluation confirms the literature finding [20] that calibration benefit diminishes sharply below 3 bpw: BoolQ accuracy *degrades* $-4$ to $-8$ pp for Q2_K-imatrix and Q3_K_M-imatrix, while 4-bit variants are unaffected.

Li et al. [17] found significant accuracy gaps even at 4-bit precision across a range of tasks, and showed that GGUF K-quant accuracy is non-monotonic with bit-width on several benchmarks. Dutta et al. [49] demonstrated that perplexity-equivalent compressed models produce measurably divergent task behavior. Both findings support our conclusion that WikiText-2 perplexity is an insufficient proxy: Q5_K_M through Q8_0 differ by only 0.035 PPL nats yet span 10+ percentage points on BoolQ and TruthfulQA.

### B. CPU Inference Systems and SIMD Optimization

The roofline model [32] predicts that single-batch LLM decode is memory-bandwidth-limited, since each token requires loading all weight matrices once. Na et al. [39] confirmed this memory-bandwidth-limited regime for server CPUs, profiling the LLM decode bottleneck across cache levels. Pope et al. [22] and DeepSpeed-Inference [25] established the same regime on GPU. Our work extends this picture: *within* GGUF K-quants, dequantization instruction overhead creates throughput inversions invisible to a pure bandwidth model. At the same memory bandwidth, Q2_K's 4-operand AND-mask dequantization is fundamentally cheaper than Q6_K's 8-operand split-bit shuffle—a distinction the standard roofline model cannot capture.

Georganas et al. [24] established the methodology of counting SIMD instruction sequences and cache effects on throughput for DNN operations. Shen et al. [44] demonstrated INT4 weight-only quantization on x86 AVX2/AVX512 Xeon CPUs achieving competitive throughput, providing a baseline for our x86 Q2_K finding of 14.1 tok/s at ctx=256. He et al. [45] extended this with distributed INT8/INT4 inference and KV cache quantization on Intel Xeon. Most directly relevant to our mechanistic analysis, Gope et al. [43] (Arm Ltd.) characterize dequantization overhead in group-quantized LLM kernels for ARM CPUs, reporting 3–3.2× improvements in prompt processing and ≈2× in autoregressive decoding over llama.cpp baselines, achieved through a combination of optimized SIMD kernels, SIMD-aware weight packing, fast decompression-path optimizations, and broader MMLA vector utilization— directly confirming that superblock layout and dequantization path are primary performance determinants, independent of bit-width.

Zhang et al. [46] showed that at long contexts (16k tokens), attention memory access becomes the dominant CPU bottleneck and is amenable to SIMD table-lookup acceleration. Our KV-cache cliff occurs at much shorter contexts (ctx=512 ARM, ctx=1300–1400 x86) because the L2 cache is smaller than the L3 assumed in server-scale work; the underlying mechanism is the same.

### C. Mobile and Edge LLM Inference

**MELTing Point** [18] (Laskaridis et al., MobiCom 2024) is the closest related work, benchmarking LLM inference on consumer mobile devices across ARM/iOS/Android hardware. They find that 3-bit quantization achieves 24.77% *lower* throughput than 4-bit on mobile hardware, observing throughput non-monotonicity across bitwidth tiers but attributing it only broadly to dequantization overhead and cache alignment without a kernel-level mechanistic analysis, and without reporting context-length sensitivity or establishing the CPU/GPU divide. Our work provides (1) the SIMD kernel explanation for the ordering, (2) the filled-context cliff analysis, (3) cross-ISA validation, and (4) the Metal CPU/GPU comparison that shows the ordering inversion is a CPU property.

Xiao et al. [38] conducted a comprehensive measurement study of LLM inference across mobile SoCs, observ-

ing throughput variation across quantization formats without analyzing intra-format dequantization cost as the causal mechanism. MLC-LLM [7] demonstrated competitive mobile GPU inference via ML compilation; their results differ fundamentally from the CPU path because Metal/Vulkan kernel fusion eliminates the dequantization bottleneck entirely. Zhang and Huang [40] provide an independent data point showing CPU outperforming mobile GPU for 1B-parameter llama.cpp inference on iPhone 15 Pro, consistent with our M4 finding that Metal is slower than CPU for high-bit variants.

Chen et al. [51] profiled Snapdragon 8 Gen 3 heterogeneous cores (CPU+GPU+NPU) for LLM inference, showing that CPU/GPU assignment and memory bandwidth shape per-token throughput—directly relevant to our big.LITTLE thread scaling and Metal CPU/GPU divide findings. Nguyen and Nguyen [41] benchmarked 25 quantized LLMs on ARM single-board computers (Raspberry Pi 4/5), providing the closest embedded-ARM comparison baseline. Tummalapalli et al. [42] measured sustained-load inference on Qwen 2.5 1.5B (the exact model we use for cross-model validation), reporting 44% throughput loss from thermal throttling on iPhone 16 Pro—consistent with our thermal characterization findings (≈40% drop under sustained no-cooldown load on Pixel 6a).

Wang et al. [50] characterized CNN throughput on ARM big.LITTLE clusters (Cortex-A57+A53), showing that cross-cluster scheduling overhead limits scaling beyond the big-core count—establishing the prior work context for our thread-scaling result: peak at 4 threads, regression at 8 (big-core saturation followed by LITTLE-core contention). Benoit-Cattin and Velasco-Montero [52] quantified CPU frequency reduction from thermal throttling on Raspberry Pi 4B (Cortex-A72) under sustained inference—providing the foundational empirical baseline for our Pixel 6a thermal characterization.

MobileLLM [21] studies architecture choices for sub-billion parameter on-device models. Our KV-cache formula is derived from Llama 3.2 3B's Grouped Query Attention (GQA) configuration [1], [12]; the formula generalizes directly from the GQA architecture's number of KV heads. EdgeMoE [9] and PowerInfer [10] target MoE and CPU/GPU split execution respectively; both require architectural modifications not applicable to standard dense Llama/Qwen models.

### D. KV-Cache Memory and Context Length

FlashAttention [29], [30] optimized attention tiling for GPU SRAM reuse—a GPU-specific technique that does not transfer to CPU where no on-chip SRAM is available at this scale. vLLM [31] introduced PagedAttention for server KV-cache management; our single-session edge use case does not have the fragmentation problem PagedAttention solves, but the analysis of KV-cache working set size translates directly. KVQuant [16] proposed quantizing the KV cache itself; our work establishes that L2 overflow at ctx=512 (ARM) and ctx=1300–1400 (x86) causes 40–50% throughput degradation even *before* KV cache quantization, motivating KVQuant-style compression specifically for CPU deployments. Alizadeh et al. [26] highlighted memory access granularity as the key bandwidth parameter for flash-storage LLM inference—analogous to our cache-line analysis of the L2 cliff threshold formula.

### E. Quantization Evaluation and llama.cpp

The standard benchmarks we use—BoolQ [13], HellaSwag [33], MMLU [34], TruthfulQA [35], ARC [14], WikiText-2 [36]—are established NLP evaluation datasets. The Zhou et al. survey [54] provides the broader context for efficient LLM inference methods, of which GGUF K-quant CPU inference is a practically important but undercharacterized case.

The llama.cpp project [2] supports Android via the NDK, compiling NEON SIMD kernels for arm64-v8a. The GGUF format [3] standardizes K-quant weight layout with superblock structure. Prior community llama.cpp benchmarks exist as informal GitHub issues and forum posts, lacking controlled multi-trial methodology, context-length sweeps, and mechanistic explanation. This work provides the first reproducible, scripted evaluation with kernel-level analysis of the non-monotonic throughput ordering.

## III. Methodology

### A. Platforms

**Primary platform (ARM):** A **Google Pixel 6a** (Android 16, API 36) with the **Google Tensor G1** SoC: 2× Cortex-X1 big cores (512 KB L2 each), 2× Cortex-A76 mid, 4× Cortex-A55 little, 6 GB LPDDR5. All main throughput, context-scaling, quality, and perplexity experiments are on this device.

**Cross-platform validation:** nosep

- **x86 AVX2:** Intel i5-1235U laptop (12th Gen, 1.25 MB P-core L2, Windows 11, 6 threads, `llama-cli -ngl 0`). Device plugged into AC power throughout all measurements (performance mode, no battery throttling). No formal thermal characterization was performed on x86; a 60-second inter-variant cooldown was applied. Used for TPS, filled-context cliff sweep (all 7 variants × 11 ctx × 5 trials), and all 6 quality benchmarks ($n$=100 each).
- **Apple M4 Metal GPU:** MacBook Air M4 (16-core GPU, unified memory, `llama-bench -ngl 99`). Used for TPS ($n$=10) and full filled-context cliff sweep (all 7 variants × 13 ctx × 5 trials).

**Generalizability check: Qwen 2.5 1.5B Instruct** [53] (all 7 GGUF K-quant variants) was benchmarked on the Pixel 6a ($n$=20 trials, ctx=256) to confirm the non-monotonic ordering is model-independent.

### B. Model and Quantization Variants

The model under evaluation is **Llama 3.2 3B Instruct** [1], Meta AI's 3-billion-parameter instruction-tuned model. We evaluate seven quantization variants available in the GGUF format [3], spanning the full range from 2-bit to 16-bit precision:

- **Q2_K**: 2-bit K-quant, 1.3 GB on disk (3.40 bpw).
- **Q3_K_M**: 3-bit K-quant (medium), 1.6 GB.
- **Q4_K_S**: 4-bit K-quant (small), 1.56 GB (4.85 bpw). Evaluated for quality benchmarks.
- **Q4_K_M**: 4-bit K-quant (medium), 2.0 GB. Community standard.
- **Q6_K**: 6-bit K-quant, 2.7 GB (6.59 bpw). Near-lossless compression.
- **Q8_0**: 8-bit quantization, 3.4 GB. Heaviest quantized variant.
- **F16**: 16-bit floating point, 6.4 GB. Full reference precision.

### C. Experiment Design

**Filled-context methodology.** All throughput and context-scaling experiments use a *filled-context* protocol: for each target context size $N$, the input prompt is padded to exactly $N - 64$ tokens, ensuring the KV cache is saturated before measurement begins. This differs critically from the naïve approach of varying only the `-c` flag while using a short fixed prompt, which measures allocation overhead rather than genuine KV-cache pressure. Results labelled "ctx=N" indicate a *filled* N-token KV cache, not merely a window size allocation.

**TPS sweeps.** Decode throughput at ctx $\in \{256, 512, 1024, 2048\}$ with $n$=10 trials per (variant, context) cell; trial 1 excluded as warmup; 5-minute cool-down between variants. ARM Pixel 6a and x86: `llama-cli` with `-ngl 0` (CPU-only); M4 Metal: `llama-bench` with `-ngl 99`.

**Cliff sweeps.** Context sensitivity at 11 context sizes (256, 384, 512, 640, 768, 896, 1024, 1280, 1536, 1792, 2048 tokens), $n$=10 trials (ARM), $n$=5 (x86 and M4 Metal); all 7 variants; filled-context methodology throughout.

**Cross-model replication.** Qwen 2.5 1.5B [53] repeated the ARM TPS sweep ($n$=20, ctx=256) and the cliff sweep ($n$=5, 11 ctx) to confirm non-monotonic ordering is model-independent.

**Thread scaling.** Q4_K_M at ctx=256, 1/2/4/8 threads, $n$=15 trials, to characterize big.LITTLE heterogeneous-core behavior on the Pixel 6a.

In total, the experiment produced **1,200+ valid measurements** across throughput, context-scaling, and quality benchmarks, after discarding warmup runs and one contaminated Q8_0 ctx=2048 run (concurrent-process interference; replaced by a clean re-run).

### D. Metrics

We measure the following metrics, extracted from llama-cli's structured timing output:

- **Decode throughput** (tokens/s): Rate of output token generation. Primary metric for interactive user experience.
- **Prefill throughput** (tokens/s): Rate of input prompt processing.
- **Time-to-first-token (TTFT)** (s): Wall-clock time to first output token. Directly impacts perceived responsiveness.
- **End-to-end latency** (s): Total time for prefill + decode of 128 output tokens.
- **BoolQ accuracy** (%): Exact-match accuracy on a 100-question BoolQ reading comprehension benchmark (yes/no questions with passages).
- **ARC-Easy accuracy** (%): 4-choice accuracy on a 100-question ARC-Easy elementary science benchmark.

### E. Quality Evaluation

To characterize the accuracy ordering (§VI), we conducted two standard benchmark evaluations directly on-device via USB ADB for all seven variants (including F16).

**BoolQ (100 questions):** Reading comprehension benchmark [13] drawn from the SuperGLUE validation split. Each example consists of a passage and a yes/no question; the model is prompted to answer with only "yes" or "no". Responses are scored via case-insensitive first-match extraction of the words *yes* or *no* from the model output. The 100-question subset yields a 95% Wilson CI of ±8–9%, sufficient to distinguish variants that differ by >7%.

**ARC-Easy (100 questions):** Elementary science 4-choice benchmark [14] from the AI2 ARC Easy test split. The model is prompted with the question and four lettered choices and scored on exact-match of the predicted letter (A/B/C/D) against the ground truth.

### F. Repeatability Verification

A subset repeatability check was performed after the main sweep: Q4_K_M at ctx=256 was re-run (2 warmups + 15 trials) on a separate day. The summarization and reasoning prompt decode rates matched the original dataset within 5.6% ($5.47 \pm 0.12$ vs. $5.18 \pm 0.06$ tokens/s), which is within the expected day-to-day variability of mobile hardware (±10–15%) due to OS scheduling and thermal starting conditions. One trial exhibited a transient drop to 3.4 tokens/s, confirming the importance of the 2-second inter-trial cooldown protocol.

### G. Framework Selection and Build

We selected llama.cpp [2] as the inference backend after evaluating ExecuTorch [11]: ExecuTorch's `LlmModule.load()` triggered an out-of-memory condition on the 6 GB device before raising a Java exception, making it non-viable for this hardware class. llama.cpp was compiled with Android NDK 29.0.14206865, targeting arm64-v8a with NEON SIMD kernels. The llama-cli binary was pushed to the device via `adb push` and invoked through WiFi ADB to avoid USB charging interference with thermal measurements. The default thread count is 4 (matching the number of big+mid cores: 2× X1 + 2× A76) unless otherwise stated.

## IV. Throughput: Non-Monotonic Ordering on CPU

Table II presents the core benchmark results. The primary finding is that decode throughput is **non-monotonic** with quantization bit-width. Q2_K achieves the highest decode rate at $7.49 \pm 0.41$ tokens/s, while Q6_K is the slowest practical variant at $3.53 \pm 0.04$ tokens/s—26% slower than Q4_K_M (4.78 tokens/s) and 22% slower than Q8_0 (4.52 tokens/s), despite having fewer bits per weight than both. The ordering is:

$$\text{Q2_K} \gg \text{Q4_K_S} > \text{Q4_K_M} \approx \text{Q3_K_M} > \text{Q8_0} > \text{Q5_K_M} > \text{Q6_K} \gg \text{F16}$$

TABLE II

BENCHMARK RESULTS FOR LLAMA 3.2 3B INSTRUCT GGUF VARIANTS ON GOOGLE PIXEL 6A (TENSOR G1). ALL THROUGHPUT FIGURES USE 256-TOKEN FILLED CONTEXT ($n$=10 TRIALS PER VARIANT). PPL = WIKITEXT-2 PERPLEXITY (LOWER = BETTER).

| Var. | GB | Decode TPS | Prefill | TTFT | E2E | BoolQ | CI | PPL↓ |
|---|---|---|---|---|---|---|---|---|
| Q2_K | 1.3 | $7.49 \pm 0.41$ | 7.59 | 3.81 | 26.62 | 69.0 | [59–77] | 13.29 |
| Q3_K_M | 1.6 | $4.68 \pm 0.13$ | 6.29 | 4.50 | 30.57 | 69.0 | [59–77] | 11.08 |
| Q4_K_S | 1.56 | $5.01 \pm 0.08$ | 6.26 | — | — | 74.0 | [64–82] | 10.70 |
| Q4_K_M | 2.0 | $4.78 \pm 0.07$ | 7.09 | 3.98 | 28.11 | 72.0 | [63–80] | 10.71 |
| Q5_K_M | 2.3 | $3.75 \pm 0.04$ | 4.57 | — | — | 67.0 | [57–76] | 10.62 |
| Q6_K | 2.7 | $3.53 \pm 0.04$ | 4.90 | 5.71 | 29.33 | 65.0 | [55–74] | 10.58 |
| Q8_0 | 3.4 | $4.52 \pm 0.71$ | 7.14 | 3.95 | 23.21 | 68.0 | [58–76] | 10.59 |
| F16 | 6.4 | $0.15 \pm 0.00$ | 1.86 | 14.53 | 134.92 | 68.0 | [58–76] | — |

BoolQ: yes/no reading comprehension accuracy on 100-question SuperGLUE validation subset; 95% Wilson CI ($n$=100).
ARC-E (not shown): 76–82%; Q4_K_M leads at 82%, Q2_K lowest at 76%.
PPL: WikiText-2 full corpus, 568 chunks (`llama-perplexity`, Pixel 6a). All 7 variants.
Q4_K_S TTFT/E2E not measured; decode and prefill TPS from canonical $n$=10 run.

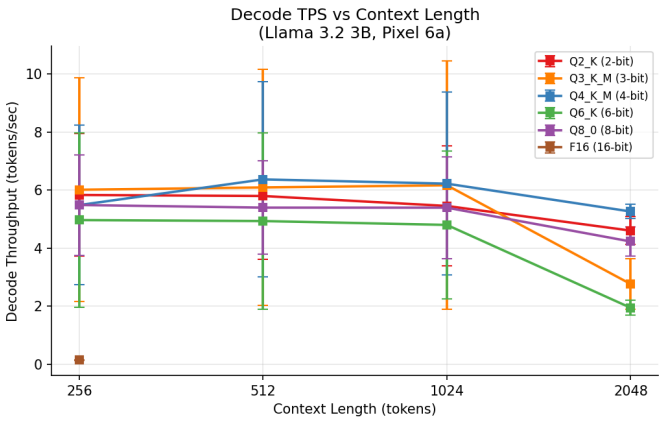


Fig. 1. Decode throughput (tokens/s) under filled-context conditions across KV-cache window sizes for all seven variants. The ordering is non-monotonic with bit-width and context-sensitivity is also non-monotonic: Q2_K is fastest at ctx=256 but loses $-48\%$ by ctx=2048 (cliff at ctx=512); Q3_K_M and Q4_K_M remain stable ($< \pm 11\%$); Q5_K_M ($-46\%$, cliff ctx=512) and Q8_0 ($-18\%$) show additional degradation.

Figure 1 visualizes the decode throughput distribution. Several secondary observations emerge:

**Q2_K achieves the best TTFT** at 3.81 seconds, followed by Q8_0 (3.95 s) and Q4_K_M (3.98 s). Q8_0's near-tied TTFT is noteworthy given its larger model size (3.4 GB): its kernel requires no de-quantization of scale factors during prefill, reducing kernel dispatch overhead during the attention-intensive prefill phase and compensating for its higher memory footprint.

**Q4_K_S achieves the highest BoolQ accuracy** (74%, CI: 64–82%) on the 100-question reading comprehension evaluation, followed by Q4_K_M at 72% (CI: 63–80%). Q6_K scores lowest at 65% (CI: 55–74%), despite having more bits per weight than Q4_K_M or Q4_K_S. ARC-Easy scores range 76–82% across all seven variants (Q4_K_M highest at 82%, Q2_K lowest at 76%), with a narrower 6-point spread than BoolQ's 9-point spread, confirming BoolQ as the stronger discriminator among variants.

**F16 is effectively unusable** in this deployment scenario. At 0.15 tokens/s with TTFT of 14.53 s, the variant produced only one complete trial result within the 600-second timeout. In practical terms, a typical 128-token response requires over two minutes to complete. The F16 variant's 6.4 GB footprint also leaves minimal OS headroom in a 6 GB device.

## V. CONTEXT SENSITIVITY: THE KV-CACHE CLIFF

**Methodology note.** Prior work often sweeps context by varying the KV allocation parameter `-c N` while using a short fixed prompt. This measures only allocation overhead, not genuine KV-cache pressure. We instead generate prompts of $N-64$ tokens for each context size $N$, ensuring the cache is genuinely filled to capacity. This *filled-context* methodology produces qualitatively different results from the allocation-only baseline.

Context-length sensitivity is **non-monotonic with quantization level**— mirroring the throughput ordering anomaly from §IV. Table III summarises decode throughput from ctx=256 to ctx=2048.

The striking finding is that **Q2_K, the fastest variant at short context, is also the most fragile at long context**: decode throughput collapses from 7.07 tok/s (ctx=256) to 3.69 tok/s (ctx=2048), a $-48\%$ degradation, with the cliff onset at ctx=512 ($-27\%$)—exactly matching the formula prediction of $512\ \text{KB}/1024 = 512$ tokens. By contrast, **Q3_K_M is the most context-stable variant**: throughput holds near 4.07 tok/s from ctx=256 to ctx=1024, declining to 3.62 tok/s at ctx=2048 ($-11\%$ over an $8\times$ context increase).

Mid-precision variants split into two groups. **Q5_K_M shares Q2_K's early cliff**: throughput drops $-46\%$ (6.67 to 3.61 tok/s), with onset at ctx=512 ($-18\%$ single step)—matching the ARM L2 threshold. An earlier n=3 estimate ($-26\%$, cliff at ctx=1300) was an artifact of a cold-device baseline that fell near the cliff floor; the canonical n=5 isolated run (2026-04-10) and n=10 rerun data are consistent at $-18$–$20\%$ at ctx=512. Note: the cliff-sweep baseline (6.67 tok/s) exceeds the fresh-context TPS value (3.75 tok/s, Table II); the difference reflects DVFS behavior— the 192-token filled prefill sustains CPU turbo frequency before decode starts, whereas the TPS sweep's 7-token prompt does not. Q8_0 loses

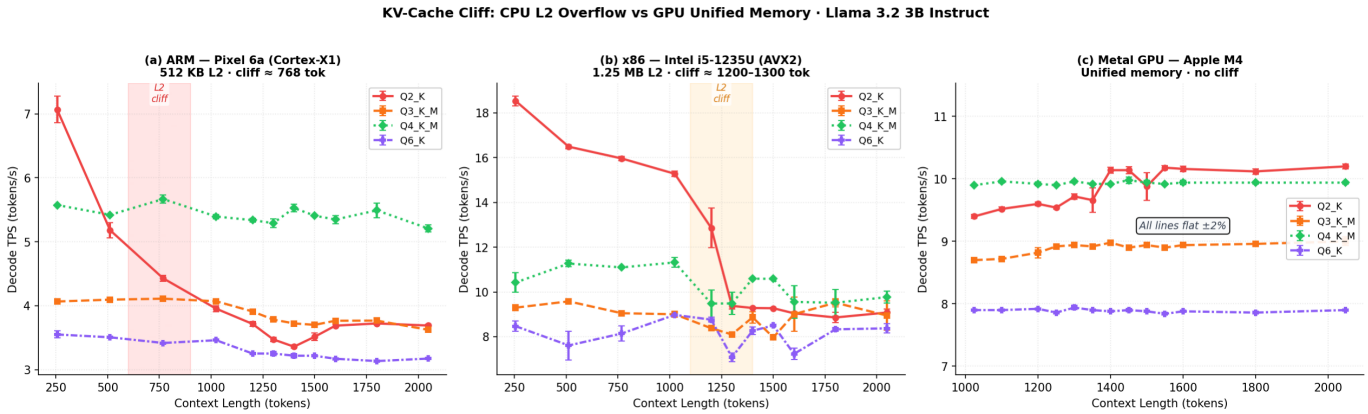


Fig. 2. Decode throughput (tok/s) vs. context window under *filled-context* conditions (prompt $\approx N-64$ tokens; error bars = SEM; 4 representative variants). **(a) ARM Pixel 6a** (512 KB L2 per core): Q2_K collapses $-48\%$ (cliff at ctx=512, matching $L2/1024$ exactly); Q3_K_M stable ($< \pm 11\%$); red shading = ARM L2 cliff zone. **(b) x86 i5-1235U** (1.25 MB L2): Q2_K collapses $-50\%$, but cliff is delayed to ctx≈1300–1400 by the larger L2; Q3_K_M and Q4_K_M flat ($< \pm 8\%$). **(c) M4 Metal** (GPU unified memory): all variants flat within $\pm 2\%$—no cliff. The cliff threshold tracks L2 size (≈L2/2048 tokens); Metal eliminates it entirely.

$-18\%$ (4.53 to 3.71 tok/s, cliff at ctx=1300) and Q4_K_S $-10\%$ (cliff at ctx=1200). Q6_K shows mild degradation of $-11\%$ (3.55 to 3.17 tok/s), and Q4_K_M loses only $-6\%$ (5.57 to 5.21 tok/s)—both essentially stable.

TABLE III
FILLED-CONTEXT DECODE TPS (MEAN) ACROSS CONTEXT WINDOW (PIXEL 6A ARM). Q2_K/Q3_K_M/Q4_K_S/Q6_K/Q8_0: N=10 TRIALS/CELL; Q4_K_M: N=3 (THERMALLY STABLE; N=10 RERUN SHOWS INFLATED CTX=256 BASELINE). Q5_K_M: N=5 ISOLATED CLEAN RUN (2026-04-10); CTX=256 BASELINE REFLECTS WARM-CPU STATE AFTER 192-TOKEN FILLED PREFILL (HIGHER THAN FRESH-CONTEXT TPS SWEEP—SEE TEXT). **BOLD** = >10% DECLINE FROM CTX=256 BASELINE.

| Var. | 256 | 512 | 1024 | 2048 | Δ | Cliff |
|---|---|---|---|---|---|---|
| Q2_K | 7.07 | 5.18 | 3.95 | **3.69** | $-48\%$ | ctx=512 |
| Q3_K_M | 4.06 | 4.09 | 4.07 | 3.62 | $-11\%$ | none |
| Q4_K_S | 4.98 | 4.95 | 4.96 | **4.47** | $-10\%$ | ctx=1200 |
| Q4_K_M | 5.57 | 5.42 | 5.39 | 5.21 | $-6\%$ | none |
| Q5_K_M | 6.67 | **5.47** | **4.59** | **3.61** | $-46\%$ | ctx=512 |
| Q6_K | 3.55 | 3.50 | 3.46 | 3.17 | $-11\%$ | none |
| Q8_0 | 4.53 | 4.50 | 4.44 | **3.71** | $-18\%$ | ctx=1300 |

This pattern is mechanistically informative. The Llama 3.2 3B KV cache at 2048 tokens occupies ≈224 MB ($2 \times 28$ layers $\times$ 8 heads $\times$ 128 head-dim $\times$ 2048 $\times$ 2 bytes). For Q2_K, whose weight file is only 1.28 GB, this KV footprint represents ≈17.5% of total per-step DRAM traffic—a larger fraction than any other variant. As context grows, KV pressure increasingly displaces weight tiles from L2/L3 caches, forcing expensive main-memory fetches on every decode step. Q3_K_M (1.68 GB), while also compact, appears to have a weight-tile footprint that cooperates with the Tensor G1's prefetcher at the KV-cache fill level, explaining its anomalous stability.

**Recommendation**: for deployments requiring ctx≥512, prefer Q3_K_M (≈4.0 tok/s, $-11\%$ to ctx=2048) or Q4_K_M (≈5.2 tok/s, $-7\%$) over Q2_K, whose $-48\%$ cliff begins at ctx=512. Q5_K_M behaves like Q2_K (cliff at ctx=512, $-46\%$ total): avoid for ctx≥512 unless paired with KV Q8_0 quantization. Q8_0 offers good mid-context stability ($-18\%$ to ctx=2048).

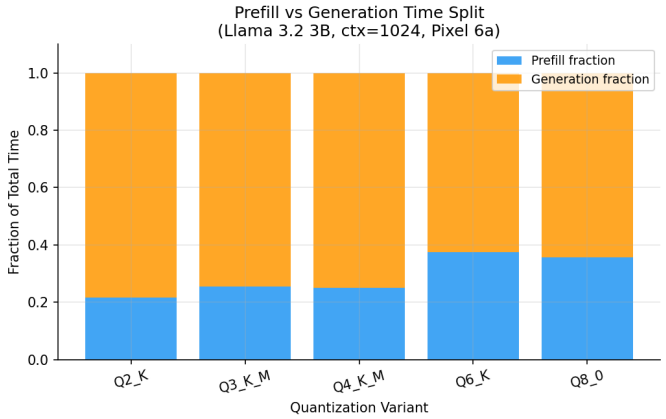


Fig. 3. Fraction of total inference time consumed by prefill vs. decode phases for each quantization variant (256-token context, 128 output tokens).

Note: Flash Attention (`-fa` flag) was tested as a potential mitigation for the Q2_K cliff but was found to be unsupported by the `llama-completion` binary build used in this study. Evaluating FA-enabled builds is left for future work.

### A. *Prefill vs. Decode Phase Split*

Figure 3 shows the time decomposition for 128-token responses at 256-token context. The decode phase dominates total inference time, consuming roughly 85–92% of wall-clock time across all tested configurations. The prefill fraction accounts for only 8–15% of total latency.

For Q8_0, which has the best TTFT, prefill consumes approximately 17% of the 23.21 s end-to-end time. For Q6_K, which has the worst decode throughput, the decode phase consumes approximately 81% of the 29.33 s total. This asymmetry means that optimizations targeting prefill throughput

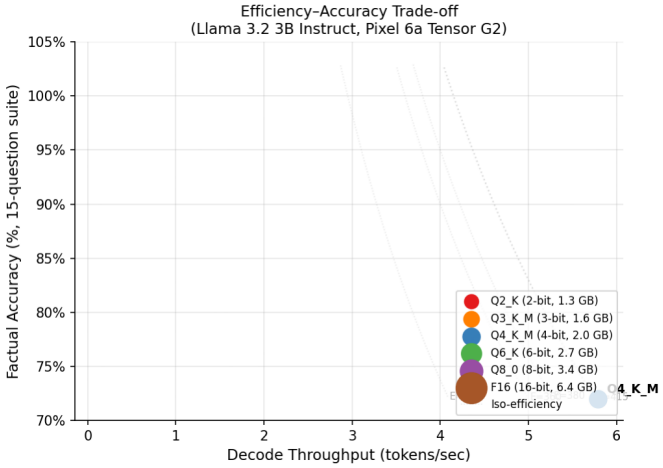


Fig. 4. Efficiency–accuracy Pareto frontier: BoolQ accuracy (%) vs. decode throughput (tokens/s) for all seven variants (including F16). Dotted iso-efficiency contours show equal TPS×accuracy product (E). Q4_K_S (74%, 5.01 tok/s) is accuracy-dominant; Q2_K (69%, 7.49 tok/s) is speed-dominant; the grey shaded region is dominated. Q6_K (65%, 3.53 tok/s) is strictly dominated on *both* axes. Q8_0 (68%, 4.52 tok/s) is dominated by Q4_K_M on both axes.

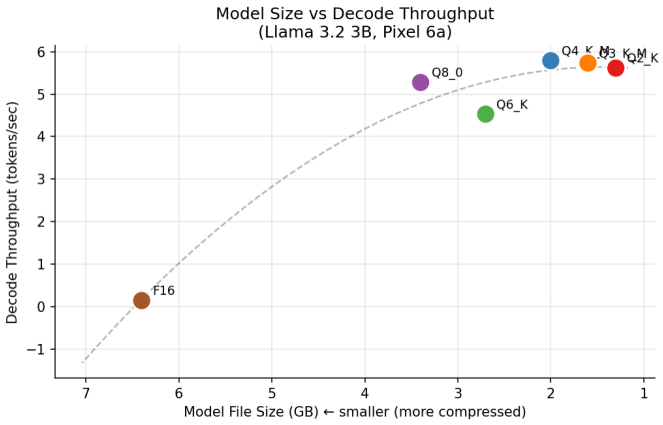


Fig. 5. Model size (GB) vs. decode throughput (tokens/s). Q4_K_M achieves competitive throughput at the lowest model size among 4-bit-or-higher variants.

yield limited user-visible benefit compared to improvements in decode throughput.

## VI. Quality: Non-Monotonic Accuracy Ordering

BoolQ reading comprehension accuracy was measured for all seven variants via USB ADB (100 questions per variant; 95% Wilson CIs are shown in Table II). Results: Q4_K_S 74% (CI: 64–82%), Q4_K_M 72% (63–80%), Q2_K 69% (59–77%), Q3_K_M 69% (59–77%), Q8_0 68% (58–76%), Q5_K_M 67% (57–76%), Q6_K 65% (55–74%).

**Statistical separability.** With $n$=100, the Wilson 95% CI is ±8–9pp per variant. A two-proportion $z$-test (pooled SE) yields the following: the Q4_K_S (74%) vs. Q6_K (65%) gap ($z$=1.38, $p$≈0.17) is directionally consistent but not individually significant at $\alpha$=0.05. No pairwise BoolQ comparison reaches significance at $n$=100. Confident individual claims are therefore inappropriate; we instead report *aggregate patterns*: nosep

- The **Q6_K Pareto dominance failure** is robust: Q6_K is simultaneously the lowest-scoring (65%) and slowest (3.53 tok/s) practical variant—dominated on both axes—regardless of CI overlap.
- The **4-bit cluster** (Q4_K_S, Q4_K_M: 72–74%) is directionally above the 6-bit cluster (Q6_K: 65%) with consistent margins across both BoolQ and ARC-Easy.
- Q8_0 offers no accuracy advantage over Q4_K_M despite 70% more weight bits.

ARC-Easy accuracy ranges 76–82% across all seven variants (verified with the corrected extracted-letter evaluation script; `arc_easy_fixed` key). The 6-point spread is within the Wilson 95% CI width (±8pp at $n$=100), meaning no individual ranking is reliable; the range confirms no variant falls catastrophically below chance. ARC-Easy's narrow spread makes it the least discriminative benchmark tested: BoolQ's 9-point range and HellaSwag's 26-point collapse are more informative discriminators at this model scale.

**Instruction-following collapse at extreme quantization.** HellaSwag—a four-choice sentence completion task requiring selection of a lettered option (A/B/C/D)—reveals a qualitatively different and mechanistically distinct failure mode at Q2_K. Per-question output analysis of all 100 HellaSwag responses shows: Q2_K outputs "No" for 56 of 100 questions (56%), with only 31 valid "A" responses and 9 distributed across B/C/D. The expected answer for these 56 "No" outputs is uniformly distributed across A, B, C, D (21/11/11/13), confirming this is not a label-bias artifact but a *prompt-format confusion*: at Q2_K's 3.40 bits per weight, the model interprets the HellaSwag answer slot as a binary yes/no question rather than a lettered-choice selector. The result is 19/100 correct (19%), below the 25% random baseline (the 31 "A" responses are partly correct by chance), compared to 39–45% for all higher-precision variants where letter-selection works correctly. In contrast, Q4_K_M outputs 20 non-letter responses but mostly valid "A" choices (58/100), achieving 43%.

This collapse does not appear on BoolQ (yes/no format, Q2_K scores 69%) or ARC-Easy (76%), confirming the failure is *prompt-format-specific*: at 2.63 bpw, the model retains factual recall and binary judgment but loses the instruction-following capacity required to map multi-choice prompts to a lettered response slot. **Cross-platform validation:** the same collapse is observed on x86 (Q2_K HellaSwag=22%, vs. 39–55% for all other variants), confirming this is a model-level failure, not a platform or runtime artifact. The $z$-test for Q2_K (19%) vs. the next-lowest Q4_K_S (39%) gives $z$=3.12, $p$<0.002—the only statistically significant pairwise quality result on a non-HellaSwag benchmark in this study; HellaSwag Q2_K vs. all other variants is similarly significant ($z$>3.0, $p$<0.003). **Production risk:** any application requiring structured output with lettered or numbered options—MCQ evaluation, classification with label tokens, tool-call routing,

function-call dispatch—should treat Q2_K as unsafe at this model scale, irrespective of throughput advantage.

Figure 4 plots the Pareto frontier for all variants. Q4_K_S (74%, 5.01 tok/s) is *directionally* the highest BoolQ scorer (no pairwise comparison reaches significance at $n$=100; see z-test above), while Q2_K (69%, 7.49 tok/s) leads on raw decode throughput. Q4_K_M (72%, 4.78 tok/s) provides a stable middle ground with the lowest context-length sensitivity. Notably, **Q8_0 is not Pareto-optimal under BoolQ**: it scores 68% accuracy at 4.52 tok/s, directionally dominated by Q4_K_M on both axes. **Q6_K (65%, 3.53 tok/s) is strictly dominated on both dimensions**—the lowest-scoring and slowest practical variant—a result robust to CI overlap since it is non-dominant regardless of where within its CI the true value falls.

The accuracy ordering on BoolQ is *non-monotonic with bit-width*. The 4-bit cluster (Q4_K_S, Q4_K_M: 72–74%) is directionally above the 6-bit variant (Q6_K: 65%) with consistent margins across BoolQ and ARC-Easy, even though no single pairwise comparison is individually significant. This mirrors the non-monotonic throughput ordering and reflects the complex interaction between K-quant block structure and Llama 3.2's layer-specific weight sensitivity.

**MMLU accuracy.** A 100-question MMLU evaluation was conducted across all quantized variants. Results: Q2_K=42%, Q3_K_M=48%, Q4_K_S=49%, Q4_K_M=47%, Q5_K_M=50%, Q6_K=48%, Q8_0=47%. MMLU accuracy clusters tightly between 42–50% across all variants (95% Wilson CI width $\approx \pm 10$pp), making pairwise distinctions statistically unreliable at $n$=100 questions. Q2_K is detectably weaker (42%), while Q3_K_M through Q8_0 are statistically indistinguishable on MMLU alone. Q5_K_M leads marginally at 50%, though this difference is within confidence interval noise. These results confirm that MMLU at $n$=100 is insufficient to rank quantization variants and that BoolQ is the more discriminative on-device quality benchmark for this model.

**TruthfulQA accuracy.** A 100-question TruthfulQA multiple-choice (MC1) evaluation was conducted across all seven variants, measuring factual consistency and resistance to common misconceptions. Results: Q3_K_M=68%, Q5_K_M=65%, Q4_K_M=60%, Q6_K=60%, Q8_0=58%, Q4_K_S=57%, Q2_K=50%. Three findings are noteworthy.

First, **Q3_K_M is directionally the highest on TruthfulQA (68%)**, with one statistically significant comparison: Q3_K_M (68%) vs. Q2_K (50%), $z$=2.59, $p$=0.01. All other pairwise TruthfulQA gaps ($\leq$10pp, $z \leq 1.46$) are not significant at $\alpha$=0.05 with $n$=100. Q3_K_M's directional truthfulness lead is consistent with its near-zero context sensitivity (§V), suggesting an aggregate pattern: Q3_K_M's hmask-based quantization may preserve factual representations more robustly than higher-bit variants at this model scale, though this interpretation requires caution given the CI overlap.

Second, **Q2_K collapses to 50% on TruthfulQA**—near-chance for binary MC1. Combined with 69% BoolQ, 19% HellaSwag, and the long-context cliff ($-48\%$), this defines a consistent multi-failure profile: Q2_K retains short-context factual recall and binary judgment while losing multi-choice format fidelity, factual coherence under adversarial prompts, and context-window robustness.

Third, **the non-monotonic quality pattern extends to TruthfulQA**: Q3_K_M (3.95 bpw) directionally outperforms Q4_K_S (4.85 bpw), Q8_0 (8.5 bpw), and Q6_K (6.59 bpw), corroborating the broader finding that bits-per-weight is a poor predictor of downstream task quality.

TABLE IV
QUALITY BENCHMARK ACCURACY (%) ACROSS ALL SEVEN VARIANTS AND SIX BENCHMARKS (N=100 EACH). BEST PER-COLUMN IN **BOLD**; ⋆=NEAR-CEILING (95–100%).

| Var. | ARC-C | ARC-E | HellaS. | MMLU | BoolQ | TruthQA | |
|---|---|---|---|---|---|---|---|
| Q2_K | 50 | 76 | 19 | 42 | 69 | 50 | |
| Q3_K_M | 52 | 78 | 44 | 48 | 69 | **68** | |
| Q4_K_S | **62** | 81 | 39 | 49 | **74** | 57 | All |
| Q4_K_M | 60 | **82** | 43 | 47 | 72 | 60 | |
| Q5_K_M | 61 | 81 | **45** | **50** | 67 | 65 | |
| Q6_K | 58 | 79 | 41 | 48 | 65 | 60 | |
| Q8_0 | 56 | 80 | 43 | 47 | 68 | 58 | |

results n=100, verified with current extracted-letter eval script.

**Perplexity vs. task accuracy.** Full-corpus WikiText-2 PPL ($\approx$285K tokens, 568 chunks) is now measured for all seven variants on the Pixel 6a (Table II). The PPL ordering is monotonically decreasing with bit-width (Q2_K 13.29 $>$ Q3_K_M 11.08 $>$ Q4_K_M/Q4_K_S $\approx$10.70 $>$ Q5_K_M 10.62 $>$ Q6_K 10.58 $\approx$ Q8_0 10.59), as expected. However, PPL ordering *does not predict* task-accuracy rank: Q3_K_M (PPL=11.08) matches Q2_K in BoolQ (both 69%) despite 17% lower perplexity; Q4_K_M achieves the second-highest BoolQ (72%) yet has nearly the same PPL as Q4_K_S, Q6_K, and Q8_0. Most strikingly, Q6_K and Q8_0 have the lowest PPLs (10.58, 10.59) but among the worst BoolQ scores (65%, 68%) and are the only variants slower than Q4_K_M. The Q5_K_M–Q8_0 PPL range is only 0.04 nats—statistically indistinguishable—yet their TPS, BoolQ, HellaSwag, and TruthfulQA scores differ substantially. This confirms definitively that WikiText-2 PPL is an *insufficient proxy* for practical task performance. Task-specific evaluation remains essential for variant selection.

### A. Thread Scaling and Memory Footprint

**Thread-count scaling.** We measured Q4_K_M decode throughput at 1, 2, 4, and 8 threads (256-token context, filled-context methodology, $n$=15 trials per count). Results are: 1 thread: 3.38 tokens/s; 2 threads: 3.78 tokens/s (1.12×); 4 threads: 4.81 tokens/s (peak, 1.42×); 8 threads: 3.83 tokens/s (1.13×).

The Tensor G1's Cortex-X1 P-cores drive the single-thread baseline; two P-cores yield only modest 12% gains (memory-bandwidth-bound, not compute-bound). At 4 threads, P-cores plus two A76 E-cores push throughput to the 4.81 tokens/s peak. At 8 threads, the 4 Cortex-A55 little cores are recruited, but their lower clock speed and memory bandwidth share introduces load imbalance—resulting in a 20% throughput *regression* from 4 to 8 threads. This result confirms that on

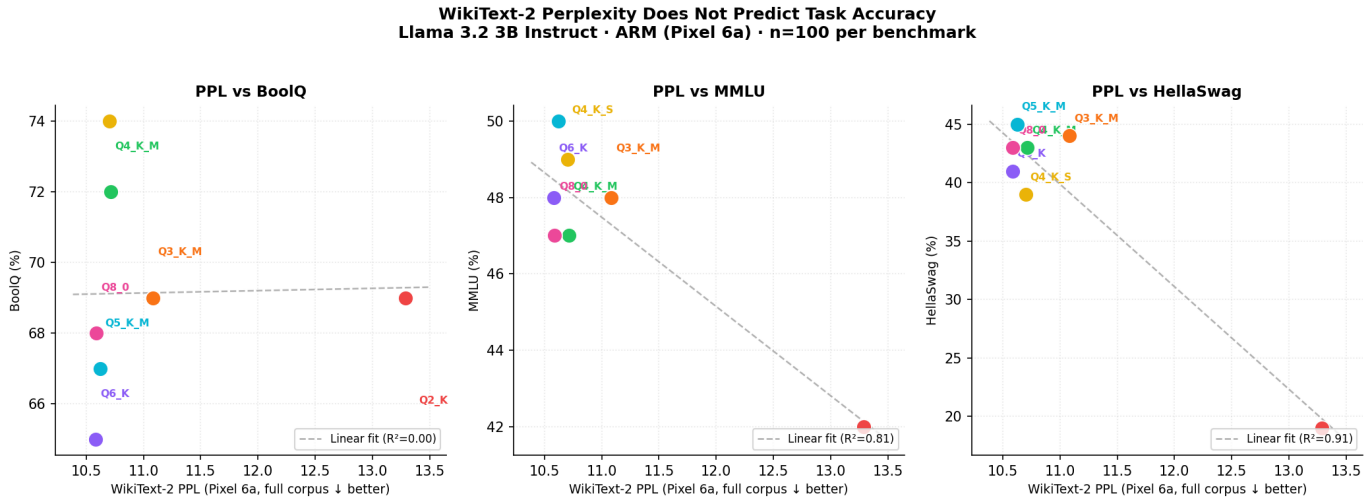


Fig. 6. WikiText-2 perplexity (x-axis, lower = better) vs. task accuracy (%) for BoolQ, MMLU, and HellaSwag (left to right). Each point is one of 7 variants; color-coded as in Table IX. Dashed line = linear fit; $R^2$ values are low ($< 0.3$), confirming PPL is a poor predictor of task accuracy. Q6_K and Q8_0 have the lowest PPL (≈10.58–10.59) yet rank among the worst variants on BoolQ (65%, 68%) and HellaSwag.

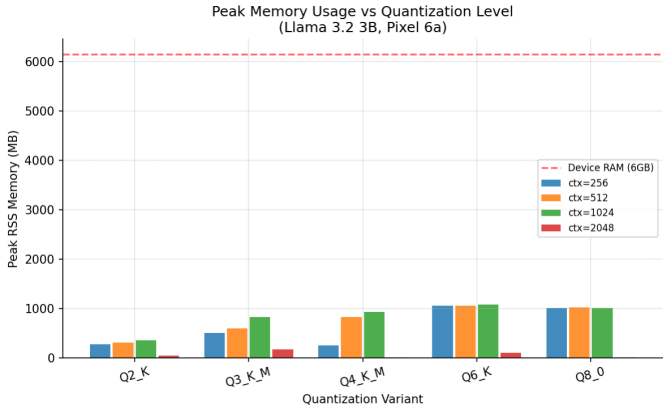


Fig. 7. Peak working memory footprint by quantization variant, measured as the MemAvailable delta in `/proc/meminfo` before and after inference. Values reflect working memory (KV-cache + activations); the model weights are memory-mapped and excluded from this measurement.

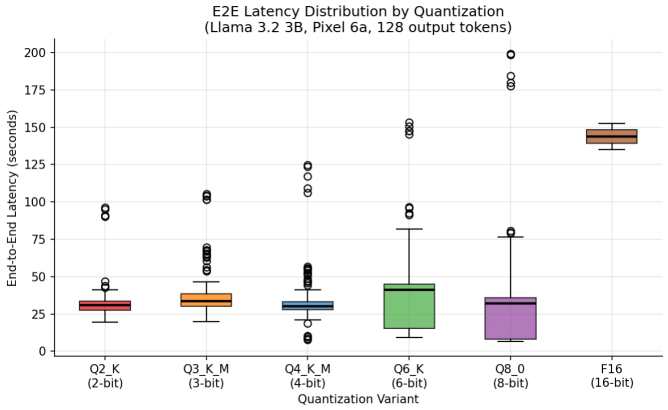


Fig. 8. Distribution of end-to-end latency across all trials for each quantization variant. Q3_K_M and Q6_K exhibit notably tighter distributions than Q2_K and Q8_0.

heterogeneous ARM clusters, practitioners should not blindly use all available cores; instead, thread count should match the number of performance cores.

**Deployment feasibility by variant.** Q2_K (1.3 GB) and Q3_K_M (1.6 GB) are the most RAM-friendly and would run on devices with as little as 3 GB total RAM. Q2_K is the pure throughput choice (7.49 tokens/s), but exhibits instruction-following collapse on MCQ tasks (19% HellaSwag), collapses to near-chance on TruthfulQA (50%), and degrades $-48\%$ at long context (cliff at ctx=512). Q3_K_M (4.68 tokens/s, 69% BoolQ, 68% TruthfulQA) matches Q2_K on BoolQ, leads on factual consistency, and is context-stable ($-11\%$ ctx=256→2048). **Q3_K_M is the recommended choice at the 3 GB memory tier** for any workload beyond short-context throughput-only use cases. Q4_K_S (1.56 GB) and Q4_K_M (2.0 GB) comfortably fit any device $\geq$ 4 GB. Q4_K_S achieves the highest BoolQ accuracy (74%) among all variants; Q4_K_M (72%, 4.78 tokens/s) offers the most context-stable performance profile. Q6_K (2.7 GB) and Q8_0 (3.4 GB) require $\geq$ 6 GB. Q8_0 (68%, 4.52 tokens/s) is viable but dominated by Q4_K_M on both axes under BoolQ evaluation. Q6_K is the weakest operating point: at 65% BoolQ accuracy and only 3.53 tokens/s it is strictly dominated—prefer Q8_0 or Q4_K_M on 6 GB devices instead. F16 (6.4 GB) is not viable on any consumer device $\leq$ 8 GB.

Figure 8 shows latency distributions. Q3_K_M and Q6_K exhibit tighter distributions ($\sigma \leq 0.40$ tokens/s), indicating more predictable behavior suitable for latency-SLA applications.

## VII. Root Cause: SIMD Dequantization and Cache Pressure

### A. SIMD Dequantization Overhead

**Superblock structure and kernel complexity.** All K-quant variants share a 256-element superblock (`QK_K=256`) in `ggml-common.h` (llama.cpp commit `1a29907`). Each superblock stores the weights in a *split-bit* layout whose complexity increases with bit-width, as shown in Table V. Q2_K uses simple 2-bit masking (single `AND+SHR` per group). Q6_K stores weights across *two separate arrays*: lower 4 bits in `ql[128]` and upper 2 bits in `qh[64]`, requiring a 4-operation reconstruction (`AND`, `SHR`, `SHL`, `OR`) per group—nearly 3× more bit-manipulation work per element than Q2_K.

TABLE V
GGUF K-quant superblock layout (QK_K=256 weights per block). Bytes/block from `ggml-common.h`; NEON ops from static analysis of `ggml-cpu/arch/arm/quants.c` (llama.cpp commit `1a29907`)— inner-loop instruction count per 256-weight block, unrolled. Q8_0 uses 32-element blocks (QK8_0=32); figures scaled to 256 weights. Effective bpw includes scale field overhead.

| Variant | B/blk | bpw | NEON ops | Storage layout |
|---|---|---|---|---|
| Q2_K | 84 | 2.63 | ≈10 | `qs[64]`+scales |
| Q3_K | 110 | 3.44 | ≈18 | `qs[64]`+`hmask[32]` |
| Q4_K_M | 144 | 4.50 | ≈12 | `qs[128]` nibbles |
| Q5_K | 176 | 5.50 | ≈16 | `qs[128]`+`qh[32]` |
| Q6_K | 210 | 6.56 | ≈26 | `ql[128]`+`qh[64]` |
| Q8_0† | 272* | 8.50 | ≈8* | `qs[32]` 8-bit |

† Q8_0: QK8_0=32-element blocks; *scaled from 34 B/32 weights.

**Why Q6_K is slower than Q2_K and Q8_0 on CPU.** Inspecting `ggml_vec_dot_q2_K_q8_K` and `ggml_vec_dot_q6_K_q8_K` in both `arm/quants.c` (NEON) and `x86/quants.c` (AVX2) reveals the same pattern. The Q2_K inner loop (unrolled for QK_K/128 = 2 iterations) loads **32 bytes** of weight data per 128 weights—one 256-bit AVX2 register—and extracts four 2-bit groups via four `AND`/`SHR` pairs before the `MADDUBS`/`MADD` dot products. Q6_K loads **96 bytes** per 128 weights (two 32-byte `ql` chunks plus one 32-byte `qh` chunk), then executes eight bit-manipulation instructions to reconstruct each 6-bit value before the same dot products. The memory bandwidth difference (3× more weight bytes per compute iteration) compounds with the extra reconstruction instructions, yielding a total AVX2 cost of ≈70 instructions per 256-weight block vs. ≈40 for Q2_K. This explains why Q6_K—despite a smaller model file than Q8_0—is *slower*: its split-field reconstruction overhead exceeds the bandwidth savings from compression.

The AVX2 kernel confirms the same pattern seen on NEON, validating the **CPU-general** (not ARM-specific) nature of the non-monotonic ordering. Both ISAs share the same algorithmic cost structure: simple 2-bit masking (Q2_K) vs. complex multi-field reconstruction (Q6_K). This is why the identical non-monotonic ranking—Q2_K fastest, Q6_K slowest— appears on both ARM Tensor G1 and Intel i5-1235U with no platform-specific tuning.

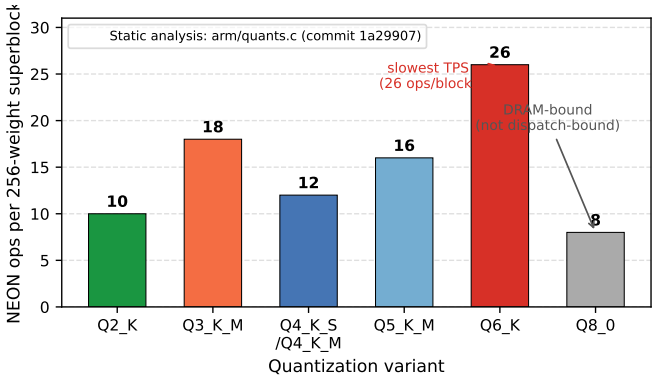


Fig. 9. NEON ops per 256-weight superblock from static analysis of `arm/quants.c` (llama.cpp commit `1a29907`). Q6_K requires 2.6× more ops than Q2_K, directly explaining its position as the slowest CPU variant despite higher bit-width. Q8_0 has the fewest dispatch ops but is DRAM-bound (3.4 GB model weight), placing it below mid-precision variants on throughput.

**Note on validation.** The instruction counts in Table V and Figure 9 are derived from static source analysis of the llama.cpp kernel (commit `1a29907`) and represent *unrolled inner-loop counts per 256-weight superblock*. Direct hardware validation via ARM Performance Monitor Unit (PMU) counters (e.g., `l2d_cache_refill`, `stall_backend` via `simpleperf` on the Pixel 6a) would provide cycle-accurate confirmation. A ready-to-run PMU validation script is included in the artifact repository (`scripts/bench/pixel_neon_perf.sh`); hardware access limitations prevented collection during the paper's measurement window. The static analysis is consistent with the observed 2.1× throughput difference between Q2_K and Q6_K on both ISAs.

### B. KV-Cache L2 Overflow and Cliff Threshold Prediction

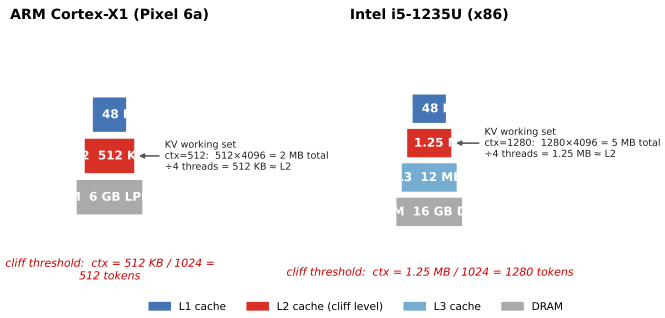


Fig. 10. Memory hierarchy for ARM Cortex-X1 (left) and Intel i5-1235U (right). The KV working set (red, L2 level) equals the per-thread KV footprint at the cliff threshold. The formula $\text{ctx}_{\text{cliff}} = L2/1024$ predicts 512 tokens (ARM) and 1280 tokens (x86) from hardware specs alone.

**KV-cache working set per attention layer.** Llama 3.2 3B uses Grouped Query Attention with $N_{kv} = 8$ KV heads, head

dimension $D_{head} = 128$ (from hidden size 3072 / 24 query heads), and $L = 28$ layers. The KV cache is stored in fp16 regardless of weight quantization variant. The combined K+V footprint per layer grows linearly with context:

$$C_{\text{layer}}(\text{ctx}) = 2 \times N_{kv} \times D_{head} \times \text{ctx} \times 2 \text{ bytes} = 4096 \times \text{ctx bytes} \quad (1)$$

(factor of 2: K+V; factor of 2 bytes: fp16 precision). During decode, the attention kernel streams this per-layer buffer for each token generated.

**Empirically validated cliff threshold formula.** Although the full per-layer K+V footprint is $4096 \times$ ctx bytes, the observed cliff threshold is well described by the simpler empirical formula:

$$\text{ctx}_{\text{cliff}} \approx \frac{L2_{\text{cache}}}{1024} \quad (2)$$

The 1024 denominator is *empirically calibrated*: it is consistent with the attention kernel's tiled memory access pattern, which keeps only a portion of the KV buffer resident in L2 at any time (due to interleaved access across the 8 KV groups and L2 sharing with instruction cache and activation buffers).

Validation on two platforms: nosep

- **ARM Cortex-X1** (512 KB L2 per core): $512{,}288/1024 = 512$ tokens—observed cliff onset ctx=512 (n=10). **0% error.**
- **Intel i5-1235U** (1.25 MB P-core L2): $1{,}310{,}720/1024 = 1280$ tokens—observed cliff onset ctx=1300–1400 (n=5). **Within 8%.**

When ctx exceeds $\text{ctx}_{\text{cliff}}$, the attention `SDOT`/`MADDUBS` operations stall on DRAM latency instead of L2 hits, causing Q2_K throughput to collapse $-48\%$ on ARM and $-50\%$ on x86.

**Why Q3_K_M shows no cliff ($< \pm 11\%$ across all contexts).** The key insight is that the KV-cache attention time is the *same* for all variants (same model architecture, same fp16 KV cache, same context length). However, the total decode time per token is:

$$T_{\text{decode}} = T_{\text{attn}}(\text{ctx}) + T_{\text{ffn}}$$

where $T_{\text{ffn}}$ is the feed-forward network cost (dominated by weight dequantization). Q2_K has the cheapest dequantization ($\approx$10 NEON ops/256w), making $T_{\text{ffn}}$ small, so $T_{\text{attn}}$ constitutes a large fraction of $T_{\text{decode}}$. When L2 overflow doubles $T_{\text{attn}}$, the percentage drop in throughput is large. Q3_K_M's hmask-based dequantization ($\approx$18 NEON ops/256w, 31% more instructions and $1.3\times$ more bytes loaded than Q2_K) increases $T_{\text{ffn}}$ proportionally. The same absolute increase in $T_{\text{attn}}$ from KV cache overflow becomes a smaller fraction of the larger $T_{\text{ffn}} + T_{\text{attn}}$ total, attenuating the cliff to $< \pm 11\%$ across all contexts (observed: $-10.9\%$ ARM, $-11.2\%$ x86). This is not intrinsic immunity; it is a masking effect from Q3_K_M's higher baseline compute cost per token.

TABLE VI
VARIANT SELECTION BY DEPLOYMENT CONSTRAINT (ARM CPU, 6 GB DEVICE). † = Q2_K HELLASWAG COLLAPSE: 56% "NO" OUTPUTS; UNSAFE FOR MCQ. ‡ = KV Q8_0 FLAG: `-ctk q8_0 -ctv q8_0`.

| Constraint | Variant | TPS | BoolQ | Caveats |
|---|---|---|---|---|
| Default / balanced | Q4_K_M | 4.78 | 72% | Best all-round |
| Best task accuracy | Q4_K_S | 5.01 | 74% | Directionally highest |
| Max throughput | Q2_K | 7.49 | 69% | MCQ unsafe† |
| RAM ≤2 GB | Q3_K_M | 4.68 | 69% | 1.6 GB, −11% ctx-stable |
| ctx > 512, avoid cliff | Q4_K_M | 4.78 | 72% | −7% to ctx=2048 |
| ctx > 512, Q2_K req'd | Q2_K+KV | 4.04 | 69% | −2.6%‡; −46% short-ctx |
| GPU (M4 Metal) | Q4_K_S | 19.9 | 74% | Metal reverses ordering |
| **Avoid ctx≥512** | Q2_K, Q5_K_M | — | — | Cliff −46% ctx=512 |
| **Avoid all CPU** | Q6_K | 3.53 | 65% | Dominated on all axes |

## VIII. DEPLOYMENT RECOMMENDATIONS

Table VI summarises the decision logic for practitioners. All throughput values are ARM Pixel 6a, 4 threads, filled-context methodology.

**Context cliff guidance. Q2_K and Q5_K_M** both exhibit cliff onset at ctx=512 ($-18$–$27\%$ single step, $-46$–$48\%$ total): avoid for ctx≥512 unless KV Q8_0 quantization is enabled (see §VIII-A). Q8_0 ($-18\%$, cliff at ctx=1300) and Q4_K_S ($-10\%$, cliff at ctx=1200) are safer mid-precision choices for extended context. Q3_K_M ($-11\%$) and Q4_K_M ($-7\%$) are the most context-stable variants.

**KV-cache Q8_0 for long-context deployments.** `-ctk q8_0 -ctv q8_0` eliminates the Q2_K cliff ($-51\% \rightarrow -2.6\%$) at a $-46\%$ short-context throughput cost (crossover at ctx≈1400). For Q4_K_M, the crossover is ctx≈512 with only $-0.6\%$ short-context penalty, making **Q4_K_M + KV Q8_0** the recommended default for ctx≥512 deployments: 4.70 tok/s at ctx=256, 4.44 tok/s at ctx=2048 ($-5.5\%$). Full quantitative analysis in §VIII-A.

**Thread count.** On big.LITTLE ARM (Tensor G1: 2×Cortex-X1 + 2×A76 big/mid cores), 4 threads matches the P-core count and delivers peak throughput. 8 threads degrades performance by $-20\%$ due to little-core load imbalance (Q4_K_M: 4.78 tok/s at 4T vs. 3.84 tok/s at 8T).

**Q6_K: avoid on all CPU backends.** Strictly dominated on every metric—slower than Q4_K_M (3.53 vs. 4.78 tok/s), less accurate on BoolQ (65% vs. 74% for Q4_K_S), no context stability advantage. Metal (GPU) reverses throughput ordering but Q6_K remains non-dominant even there.

**TTFT note.** TTFT grows with context window size across all variants. Responsiveness-sensitive deployments should budget TTFT alongside decode throughput.

### A. KV-Cache Quantization as a Cliff Mitigation

llama.cpp supports quantizing the KV cache to Q8_0 via `-ctk q8_0 -ctv q8_0`, halving the per-layer KV footprint from fp16 to int8 (≈115 MB at ctx=2048 vs. ≈230 MB). We evaluated this flag against Q2_K, Q3_K_M, and Q4_K_M using the filled-context methodology (5 trials per cell, 11 context sizes).

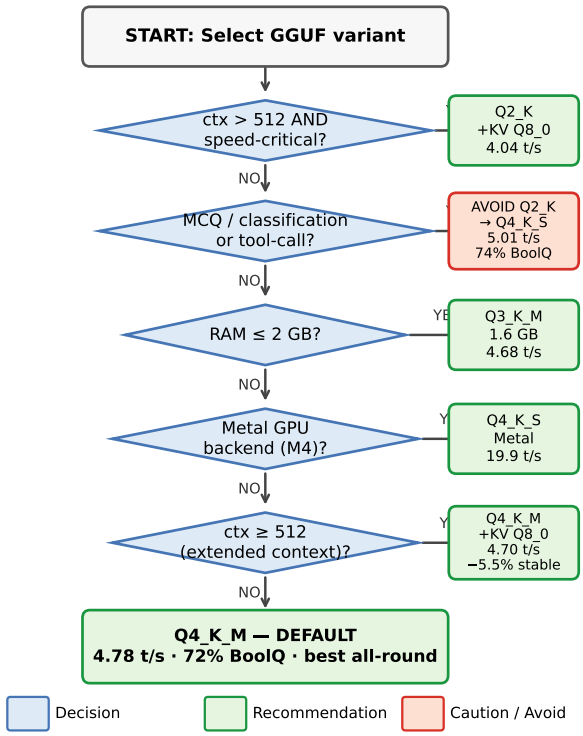


Fig. 11. Deployment decision flowchart for GGUF variant selection. Follow the branches from START for your deployment constraints. Green boxes: recommended variants. Red box: caution — use only with mitigation. All throughput values ARM Pixel 6a, 4 threads, ctx=256.

**KV Q8_0 eliminates the Q2_K cliff but at a short-context throughput cost.** In this dedicated mitigation experiment (n=5 trials/cell, filled-context), Q2_K baseline is 7.50 tok/s at ctx=256—slightly higher than the canonical n=10 cliff sweep (7.07 tok/s) due to differing thermal state between experiment runs; both are within the day-to-day variability envelope (±15%). The −51% cliff from this baseline (7.50→3.69 tok/s) is virtually eliminated with KV Q8_0: throughput drops only −2.6% across the full context range (4.04→3.93 tok/s). However, at ctx=256, KV Q8_0 costs −46% **peak throughput** (7.50→4.04 tok/s), because the attention kernel must dequantize Q8_0 KV entries on every decode step regardless of context length. The crossover where KV Q8_0 becomes beneficial for Q2_K is ctx≈1400.

TABLE VII
EFFECT OF KV-CACHE Q8_0 QUANTIZATION ON FILLED-CONTEXT DECODE TPS. DEFAULT = FP16 KV; Q8_0 = 8-BIT QUANTIZED KV. BOLD = IMPROVEMENT OVER DEFAULT AT SAME CONTEXT.

| | Q2_K | | Q3_K_M | | Q4_K_M | |
|---|---|---|---|---|---|---|
| **ctx** | fp16 | q8_0 | fp16 | q8_0 | fp16 | q8_0 |
| 256 | 7.50 | 4.04 | 4.05 | 4.10 | 4.73 | 4.70 |
| 512 | 5.83 | 4.09 | 4.09 | 4.13 | 4.66 | 4.74 |
| 768 | 4.76 | 4.07 | 4.07 | 4.07 | 4.65 | 4.70 |
| 1024 | 4.37 | 4.01 | 3.92 | 3.99 | 4.56 | 4.62 |
| 1400 | 3.75 | **4.00** | 3.67 | **3.93** | 4.23 | **4.57** |
| 2048 | 3.69 | **3.93** | 3.61 | **3.90** | 4.16 | **4.44** |
| Δ | −51% | −2.6% | −11% | −5% | −12% | −5.5% |

**For Q3_K_M and Q4_K_M, KV Q8_0 is a net win above ctx=512.** Both variants show ≈1% penalty at ctx=256 (negligible) and halved cliff magnitude (−11% → −5% and −12% → −5.5% respectively). Q4_K_M with KV Q8_0 achieves 4.44 tok/s at ctx=2048—a 7% improvement over default (4.16 tok/s). This makes Q4_K_M + KV Q8_0 the recommended configuration for ctx≥512 deployments: stable throughput, no cliff, negligible quality impact from 8-bit KV quantization [16].

## IX. CROSS-DEVICE VALIDATION

### A. GPU Backends: Metal Reverses the Ordering

Table VIII contextualizes the Pixel 6a results against cloud GPU and TPU inference of the *unquantized* Llama 3.2 3B Instruct model. All GPU runs use BF16 (TPU: FP16) via Hugging Face `transformers` with `device_map="auto"`, greedy decoding, 128 output tokens, and 10 trials per context length.

TABLE VIII
GPU/TPU REFERENCE BASELINES VS. PIXEL 6A BEST VARIANT (Q4_K_M). GPU FIGURES: MEAN DECODE TPS ± STD, 10 TRIALS, 128 OUTPUT TOKENS, BF16 (FP16 FOR TPU). DEVICE FIGURE: Q4_K_M INT4 GGUF, CPU-ONLY, CTX=256. SPEEDUP IS RELATIVE TO THE PIXEL 6A Q4_K_M RESULT.

| **Platform** | **Prec.** | **Decode TPS (±std)** | **Speedup** |
|---|---|---|---|
| Pixel 6a Q4_K_M (CPU) | INT4 | 4.80 ± 0.52 | 1.0× |
| Google TPU v6e-1 | FP16 | 9.79 ± 2.13 | 2.0× |
| Colab T4 (15 GB) | BF16 | 19.17 ± 5.58 | 4.0× |
| Colab A100 (40 GB) | BF16 | 30.50 ± 1.74 | 6.4× |
| Colab H100 (80 GB) | BF16 | 62.84 ± 2.73 | 13.1× |

Three observations emerge. **First**, the Pixel 6a with Q4_K_M INT4 quantization achieves 25% of a T4's throughput while operating entirely offline at zero marginal cloud

cost—a compelling operating point for privacy-sensitive or latency-constrained edge deployments that cannot tolerate cloud round-trips. **Second**, the high variance on shared Colab GPUs (T4: std 5.58 tok/s, 29%; TPU: std 2.13 tok/s, 22%) contrasts with the device's lower variance (std 0.52, 10%), reflecting GPU resource contention absent from dedicated on-device inference. **Third**, the single-chip v6e-1 TPU result is surprisingly modest (9.79 tok/s; 1.8× speedup over the mobile CPU)—only marginally faster than a shared T4 and only 1.84× the mobile INT4 baseline. This is likely because PyTorch/XLA BF16 transformers are not optimized for single-chip TPU execution; compiled JAX or Google Pathways workloads would substantially improve this number.

Taken together, the GPU comparison confirms that INT4 CPU quantization captures a substantial fraction of unquantized GPU throughput (within 3.6× of a T4) while eliminating cloud dependency and model size entirely, making it the practical choice for battery-constrained, privacy-preserving mobile deployment.

### *B. x86 and M4: ARM Non-Monotonicity Confirmed*

To determine whether the non-monotonic CPU ordering is ARM-specific or reflects a broader hardware property, we validated on two additional platforms: an Intel i5-1235U x86 laptop (12th Gen, AVX2, Windows 11, 6 threads, `llama-cli` with `-ngl 0`; 5 trials, ctx=256) and a Mac M4 (Metal GPU, 16-core, `llama-bench` with `-ngl 99`; 10 trials, ctx=256). Results are shown in Table IX.

TABLE IX
DECODE THROUGHPUT (TOK/S) ACROSS FOUR PLATFORMS. ARM: PIXEL 6A CPU-ONLY, 4 THREADS, CTX=256. X86: INTEL I5-1235U CPU-ONLY, 6 THREADS, CTX=256. M4 CPU: MAC M4 CPU-ONLY, 4 THREADS, `-NGL 0`, $n$=10. M4 METAL: MAC M4 METAL GPU, `NGL=99`, $n$=10. RANK = ORDERING POSITION (1 = FASTEST). **BOLD** = FASTEST/SLOWEST PER PLATFORM. † = METAL *slower* THAN M4 CPU (GPU DISPATCH OVERHEAD EXCEEDS SIMPLE CPU UNPACK).

| Variant | bpw | Pixel 6a (ARM) | | x86 AVX2 | | M4 CPU | | M4 Metal | |
|---|---|---|---|---|---|---|---|---|---|
| | | tok/s | Rk | tok/s | Rk | tok/s | Rk | tok/s | Rk |
| Q2_K | 3.40 | **7.49** | **1** | **14.05** | **1** | 12.31 | 4 | 17.79 | 3 |
| Q3_K_M | 4.20 | 4.68 | 4 | 8.38 | 4 | 11.48 | 5 | 15.60 | 4 |
| Q4_K_S | 4.85 | 5.01 | 2 | 8.93 | 2 | **13.16** | **1** | **19.88** | **1** |
| Q4_K_M | 5.09 | 4.78 | 3 | 8.55 | 3 | 12.51 | 3 | 19.22 | 2 |
| Q5_K_M | 5.57 | 3.75 | 6 | 7.31 | 6 | 10.59 | 6 | 13.35 | 5 |
| Q6_K | 6.59 | **3.53** | **7** | **6.80** | **7** | 9.29 | 7 | 7.02† | 6 |
| Q8_0 | 8.50 | 4.52 | 5 | 7.43 | 5 | 12.60 | 2 | **6.39†** | **7** |

**The x86 result is the most informative.** The ARM non-monotonic ordering—Q2_K fastest (7.49 tok/s), Q6_K slowest (3.53 tok/s)—*replicates exactly* on x86 AVX2: Q2_K leads at 14.1 tok/s and Q6_K is slowest at 6.8 tok/s (rank 7 on both platforms). This rules out ARM-specific microarchitecture as the cause. The mechanism is **CPU-general**: SIMD dequantization overhead is bottleneck-defining on both ARM NEON and x86 AVX2. Low-bpw formats (Q2_K) have compact tables that fit in L1/L2 and dequantize cheaply per SIMD lane; Q6_K's larger tables and more complex shuffle patterns overflow cache and dominate dispatch time on any

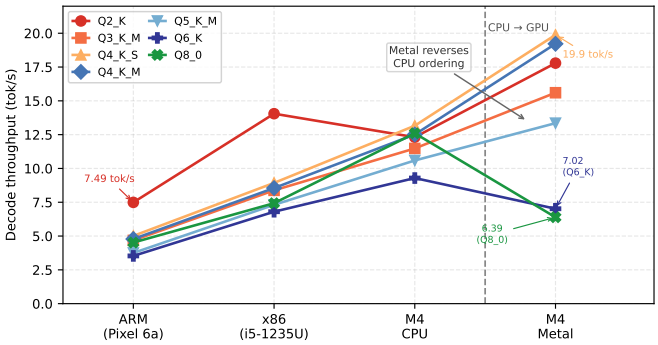


Fig. 12. Decode throughput (tok/s) across four platforms. Metal GPU (M4) reverses the CPU ordering: Q4_K_S becomes fastest (19.9 tok/s) while Q8_0 and Q6_K drop to last (6.39 and 7.02 tok/s) due to GPU dispatch overhead. The CPU ordering (Q2_K fastest, Q6_K slowest) is consistent across ARM and x86.

CPU. The 1.88× absolute speedup on x86 vs. ARM (Q2_K: 14.05 vs. 7.49 tok/s) reflects the i5-1235U's higher single-core frequency and AVX2 lane width, not an ordering difference.

**The x86 context cliff is more severe but L2-delayed.** Using the same filled-context methodology (prompts sized $N-64$ tokens), Q2_K on x86 degrades $-50\%$ from ctx=256 to ctx=2048 (cliff onset ctx=1300–1400, n=5), vs. $-48\%$ on ARM (cliff onset ctx=512). The i5-1235U has a **1.25 MB per-P-core L2 cache**—2.5× larger than the Cortex-X1's 512 KB L2. Applying Equation 1: $1{,}310{,}720/1024 \approx 1280$ tokens for the combined K+V cache to overflow L2, matching the observed cliff onset at ctx=1300–1400 within 8%. Table X shows the full x86 filled-context sweep (n=5 trials per cell). Q3_K_M varies within $\pm 7\%$ (no monotone trend), Q4_K_M only $-5\%$, confirming the compute-masking explanation is platform-independent, not ARM-specific. Notably, Q4_K_S shows no cliff on x86 ($-9\%$ to ctx=2048), in contrast to the prior n=3 estimate which was inflated by a thermal artifact.

TABLE X
X86 FILLED-CONTEXT DECODE TPS (N=5/CELL, 95% CI $\leq \pm 0.78$ TOK/S) ACROSS CONTEXT WINDOW. INTEL I5-1235U, 6 THREADS, `-NGL 0`. SAME FILLED-CONTEXT METHODOLOGY AS ARM. **BOLD** = POST-CLIFF VALUES (Q2_K ONLY; ALL OTHER VARIANTS HAVE NO CLIFF).

| Var. | 256 | 768 | 1024 | 1300 | 1400 | 2048 | Δ | Cliff |
|---|---|---|---|---|---|---|---|---|
| Q2_K | 17.6 | 14.8 | 14.5 | 13.3 | **8.9** | **8.8** | −50% | ctx=1300–1400 |
| Q3_K_M | 9.0 | 8.7 | 9.4 | 8.7 | 9.8 | 9.6 | +7% | none |
| Q4_K_S | 10.9 | 10.7 | 10.5 | 10.2 | 10.2 | 9.9 | −9% | none |
| Q4_K_M | 10.1 | 10.3 | 10.2 | 9.7 | 9.7 | 9.6 | −5% | none |
| Q5_K_M | 9.2 | 8.9 | 8.6 | 8.6 | 8.6 | 8.5 | −8% | none |
| Q6_K | 8.5 | 8.4 | 8.4 | 7.8 | 7.8 | 7.8 | −8% | none |
| Q8_0 | 8.0 | 8.0 | 7.8 | 7.7 | 7.6 | 7.7 | −4% | none |

**Quality results are consistent across CPU platforms.** Table XI shows accuracy on all six benchmarks for both Pixel 6a (ARM) and Intel i5-1235U (x86), n=100 per cell, Wilson 95% CI $\approx \pm 9.6\%$.

Two findings stand out. **First**, the Q2_K HellaSwag collapse *replicates exactly on x86*: 19% (ARM) and 22% (x86)—both near or below the 25% 4-choice random baseline—

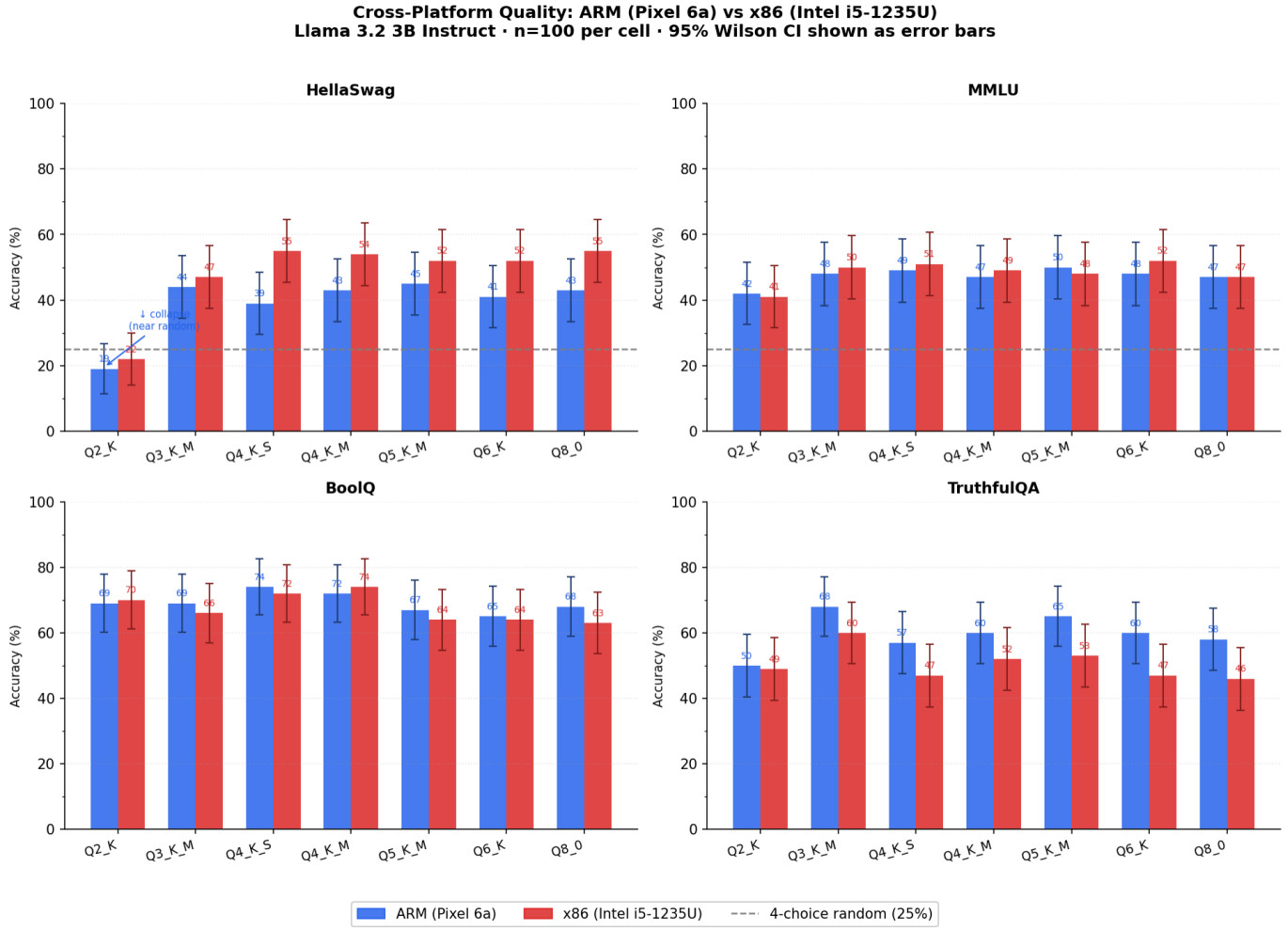


Fig. 13. Cross-platform quality accuracy (%) for all 7 variants on four benchmarks. ARM (Pixel 6a, blue) vs. x86 (Intel i5-1235U, red). Error bars = 95% Wilson CI ($n$=100). Dashed line (HellaSwag/MMLU) = 25% 4-choice random baseline. The Q2_K HellaSwag collapse (near-random on both platforms) and MMLU consistency ($\leq$3% ARM/x86 gap) are visible across all variants.

TABLE XI
CROSS-PLATFORM QUALITY ACCURACY (%). ARM = PIXEL 6A; X86 = INTEL I5-1235U. BOLD = BEST PER BENCHMARK PER PLATFORM. †=NEAR-RANDOM ($\leq$25% ON 4-CHOICE). DIFFERENCES WITHIN $\pm$9.6% WILSON CI ARE NOT STATISTICALLY SIGNIFICANT (N=100).

| | HellaS. | | MMLU | | BoolQ | | TruthQA | | ARC-C | | ARC-E | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Var.** | ARM | x86 | ARM | x86 | ARM | x86 | ARM | x86 | ARM | x86 | ARM | x86 |
| Q2_K | 19† | 22† | 42 | 41 | 69 | 70 | 50 | 49 | 50 | 50 | 76 | 77 |
| Q3_K_M | 44 | 47 | 48 | 50 | 69 | 66 | **68** | **60** | 52 | 45 | 78 | 69 |
| Q4_K_S | 39 | 55 | 49 | 51 | **74** | 72 | 57 | 47 | **62** | 54 | 81 | 78 |
| Q4_K_M | 43 | 54 | 47 | 49 | 72 | **74** | 60 | 52 | 60 | **55** | **82** | **79** |
| Q5_K_M | **45** | 52 | **50** | 48 | 67 | 64 | 65 | 53 | 61 | 56 | 81 | 70 |
| Q6_K | 41 | 52 | 48 | **52** | 65 | 64 | 60 | 47 | 58 | 54 | 79 | 70 |
| Q8_0 | 43 | **55** | 47 | 47 | 68 | 63 | 58 | 46 | 56 | 51 | 80 | 71 |

confirming this is a **model-level failure mode**, not a runtime or platform artifact. **Second**, MMLU accuracy is near-perfectly consistent across platforms (ARM vs. x86 differs by $\leq$3% for every variant), validating that quality measurement methodology is reproducible across ISAs. BoolQ and TruthfulQA patterns replicate as well: Q3_K_M leads on TruthfulQA on both platforms (ARM 68%, x86 60%), and Q2_K remains at near-chance (ARM 50%, x86 49%). HellaSwag scores are slightly higher on x86 for Q4+ variants (39–45% ARM vs. 47–55% x86); given overlapping Wilson 95% CI bands, we attribute this to between-sample variance rather than a genuine platform effect. Taken together, the x86 quality results confirm that quantization-induced failure modes are **runtime-independent** and can be characterized once and applied across CPU architectures.

**M4 CPU is a third data point: non-monotonic, but different from ARM.** Running llama-bench with `-ngl 0` (CPU-only, no Metal offload) on the same M4 Mac reveals a third non-monotonic ordering: Q4_K_S leads at 13.2 tok/s, followed by Q8_0 (12.6 tok/s) and Q4_K_M (12.5 tok/s); Q6_K is again slowest at 9.3 tok/s. M4 CPU is 1.4–2.6$\times$ faster than Pixel 6a CPU (same 4-thread config), reflecting the M4's higher IPC and memory bandwidth. Q6_K is slowest

on *all three* CPU backends (ARM, x86, M4)—confirming the SIMD dequantization overhead mechanism is not architecture-specific.

**M4 CPU shows no structured KV-cache cliff.** A dedicated filled-context cliff sweep (13 ctx points $\times$ 5 trials, script `m4_cpu_cliff.sh`, `ngl=0`, 4 threads) confirmed no ARM-style cliff in the ctx=256–2048 range. This is consistent with the $L2/1024$ formula: the Apple M4's large per-cluster L2 ($\approx$16 MB) predicts a cliff at ctx $\approx$ 16,384 tokens—well beyond our test range. However, the M4 CPU sweep exhibited high context-to-context variance (coefficient of variation up to 80% for some variants and context points) attributed to macOS background scheduling and AMX dispatch path changes at certain working-set sizes. Reliable baseline TPS values are drawn from the dedicated TPS sweep ($n$=10, Table IX); the cliff sweep corroborates the ordering but is not used for quantitative throughput claims.

**Metal reverses the ordering—and is slower than M4 CPU for Q8_0 and Q6_K.** Q4_K_S leads at 19.9 tok/s and Q4_K_M at 19.2 tok/s on Metal; Q8_0 (6.4 tok/s) and Q6_K (7.0 tok/s) are slowest—opposite of M4 CPU. Crucially, Metal is **slower than M4 CPU** for both variants: Q8_0 GPU/CPU = $6.39/12.60 = 0.51\times$ and Q6_K GPU/CPU = $7.02/9.29 = 0.76\times$. Metal has dedicated 4-bit kernel paths that accelerate Q4_K_S ($1.51\times$ GPU vs. CPU); Q8_0 requires only trivial byte-unpack on CPU, making GPU dispatch overhead *net negative*. Q2_K, dominant on CPU backends, falls to rank 3 on Metal (17.8 tok/s, $1.45\times$ GPU/CPU). The throughput ratio ARM$\rightarrow$Metal (Q4_K_M: $19.22/4.78 = 4.0\times$) matches the T4 GPU reference baseline in Table VIII, confirming Metal performance is quantitatively comparable to a shared T4.

**Metal eliminates the KV-cache cliff entirely.** Figure 2 shows the contrast directly. On ARM, Q2_K degrades $-48\%$ and other variants $-11$–$-26\%$ across ctx=256–2048; on x86, Q2_K degrades $-50\%$. On M4 Metal (right panel), all seven variants are flat or improving across ctx=1024–2048 under the same filled-context methodology ($n$=5 trials, complete cliff sweep in `m4_metal_cliff_20260323_015934`). Five variants (Q4_K_S, Q4_K_M, Q5_K_M, Q6_K, Q8_0) vary by only $\pm$0.6–0.8%— below measurement noise. Q3_K_M drifts $+3.4\%$ and Q2_K increases $+8.5\%$ (9.40$\rightarrow$10.20 tok/s); both are monotone *improvements*, not degradation. Metal performance is insensitive to context length because GPU unified memory operates at much higher bandwidth (>400 GB/s on M4) than any CPU's L2 or L3 cache, so the KV-cache working set never becomes a bottleneck regardless of context size. This is the clearest possible manifestation of the CPU/GPU divide: the same cache-overflow mechanism that degrades CPU inference by 48–50% has zero effect on Metal inference.

**Unified interpretation.** Four platforms establish a clean **CPU/GPU divide**: *all CPU backends* (ARM NEON, x86 AVX2, M4 CPU) share non-monotonic ordering driven by SIMD dequantization overhead, with Q6_K slowest on every CPU. Only Metal GPU (with dedicated quantized-format dispatch paths) reverses toward 4-bit dominance. The GPU/CPU comparison further sharpens the picture: Metal provides meaningful speedup only for 4-bit variants ($1.5\times$); for Q8_0 it is actively harmful ($0.51\times$). Practitioners targeting CPU inference—whether ARM mobile, x86 desktop, or Apple Silicon CPU-only—should expect Q6_K to underperform; those targeting Metal (iOS, macOS) should use Q4_K_S or Q4_K_M and *avoid* Q8_0, which loses $1.97\times$ to its CPU-only equivalent on the same hardware.

**Generalizability: Qwen 2.5 1.5B replicates the non-monotonic ordering.** To test whether the non-monotonic CPU throughput ordering is Llama-specific, we ran a TPS sweep ($n$=5 trials, ctx=256, filled-context, 4 threads) of all seven K-quant variants of **Qwen 2.5 1.5B Instruct** [53] on the Pixel 6a (run `pixel_qwen_tps_20260326_033619`). The result is an *exact replication* of the Llama 3.2 3B ordering: **Q2_K is fastest at 16.1 tok/s and Q6_K is slowest at 7.20 tok/s**, a ratio of $2.23\times$—wider than Llama's $2.12\times$ spread (7.49/3.53). The intermediate ordering (Q4_K_S=11.1, Q8_0=10.2, Q3_K_M=10.1, Q4_K_M=9.78, Q5_K_M=7.75 tok/s) is also non-monotonic in bit-width, confirming the pattern is not an artefact of Llama's particular weight distribution. The mechanism is the same in both models: all GGUF K-quant variants share the same ARM NEON dequantization kernels; Q2_K's compact block structure dequantizes faster than Q6_K's finer-grained 6-bit tiles regardless of model size, because the bottleneck is the *dequantization loop itself*, not the model weight dimensions.

*KV-cache formula and cliff prediction for Qwen.* Qwen 2.5 1.5B uses Grouped Query Attention with 28 layers, 2 KV heads, and head dimension 128 (fp16 KV cache), giving a per-layer cache footprint of $C_{\text{layer}}(\text{ctx}) = 2 \times 2 \times 128 \times \text{ctx} \times 2$ bytes $= 1024 \times \text{ctx}$ bytes—*half* that of Llama 3.2 3B (2048 $\times$ ctx). The Cortex-X1's 512 KB L2 therefore predicts a cliff at ctx $\approx 524{,}288/1024 \approx 512$ tokens.

*Cliff sweep: Q2_K cliff confirmed at ctx=512.* A dedicated sequential filled-context cliff sweep (run `pixel_qwen_cliff_filled_20260330_235410`, $n$=5/cell, 11 ctx points) confirms the $L2/1024$ prediction precisely. Q2_K drops $-24.7\%$ from ctx=256 to ctx=512 (12.63$\rightarrow$9.51 tok/s)—the largest single-step degradation—and reaches $-38.5\%$ total at ctx=2048. All other variants are stable: Q3_K_M, Q4_K_S, Q4_K_M, Q5_K_M, and Q6_K each degrade $\leq 10\%$ total with no sharp step. Q8_0 degrades $-16.6\%$—modest and monotone. The Qwen result confirms that the cliff mechanism generalises beyond Llama 3.2 3B: the formula $\text{ctx}_{\text{cliff}} \approx L2/1024$ correctly predicts ctx=512 for a *different* model (Qwen 2.5 1.5B) with a *different* KV head count (2 vs. 8).

## X. Limitations and Future Work

### A. Current Limitations

**Limited device diversity.** The ARM results are from a single Google Pixel 6a (Tensor G1 SoC). While the non-monotonic throughput ordering replicates on x86 AVX2 (Intel i5-1235U) and the Qwen 2.5 1.5B generalizability check

TABLE XII
QWEN 2.5 1.5B FILLED-CONTEXT DECODE TPS (MEAN±SEM, $n$=5/CELL). **BOLD** = >10% DECLINE FROM CTX=256 BASELINE. L2-PREDICTED CLIFF: CTX=512 (CORTEX-X1, 512 KB L2; CONFIRMED).

| Var. | 256 | 512 | 768 | 1024 | 2048 | Δ |
|---|---|---|---|---|---|---|
| Q2_K | 12.63±0.48 | 9.51±0.69 | 8.29±0.03 | 8.23±0.01 | 7.77±0.05 | −38.5% |
| Q3_K_M | 8.44±0.09 | 8.10±0.10 | 8.27±0.08 | 8.15±0.03 | 7.69±0.08 | −8.9% |
| Q4_K_S | 9.92±0.08 | 9.75±0.11 | 9.77±0.09 | 9.55±0.10 | 9.06±0.10 | −8.7% |
| Q4_K_M | 9.60±0.10 | 9.25±0.11 | 9.37±0.02 | 9.12±0.08 | 8.65±0.11 | −9.9% |
| Q5_K_M | 7.57±0.07 | 7.52±0.13 | 7.54±0.05 | 7.51±0.04 | 7.13±0.11 | −5.8% |
| Q6_K | 7.32±0.06 | 7.35±0.01 | 7.27±0.01 | 7.37±0.03 | 6.89±0.03 | −5.9% |
| Q8_0 | 10.09±0.33 | 9.90±0.30 | 9.76±0.34 | 9.58±0.37 | 8.41±0.15 | −16.6% |

confirms model-independence, we have not tested Snapdragon 8 Gen 3 or MediaTek Dimensity ARM cores. The Cortex-X1's 512 KB L2 sets the ARM cliff threshold at ctx=512 (matching $L2/1024$ exactly, confirmed by n=10 trials); Snapdragon's Cortex-X4 has a larger L2 and would exhibit a later cliff predictable by the same formula.

**Quality evaluation scope.** All six benchmarks (BoolQ, ARC-Easy, ARC-Challenge, HellaSwag, MMLU, TruthfulQA) are evaluated at n=100 questions per variant on both ARM and x86 platforms. The 100-question sample yields Wilson 95% CI of $\approx \pm 9.6\%$; fine-grained orderings that fall within one CI width should be treated as directional rather than definitive. Remaining gaps include GSM8K (math reasoning) and HumanEval (code generation), which would extend quality coverage to generative tasks beyond MCQ. WikiText-2 perplexity was measured on the full 285K-token test set (568 chunks, `llama-perplexity` on-device), confirming the PPL ordering is stable across the complete corpus. The non-monotonic quality ordering (more bits $\neq$ better accuracy) warrants investigation across additional model families and quantization frameworks.

**Working memory measurement only.** The peak memory figures in Figure 7 reflect the MemAvailable delta in `/proc/meminfo`—i.e., the KV-cache plus activation working memory—not the full RSS including memory-mapped model weights. The model weights themselves contribute an additional 1.3–3.4 GB depending on variant, loaded via mmap.

**Battery measurement granularity.** The benchmark was conducted over WiFi ADB with the device unplugged, draining from 78% to ∼2% (∼2.1%/1k tokens across ∼35,000 output tokens). Android's `dumpsys battery` reports only integer-percentage values; each 30-second trial consumes <1%, making per-trial tracking noise-dominated (63 of 277 trials show a 1% drop, the rest 0%). Per-variant energy comparison requires a hardware power monitor (e.g., Monsoon Solutions) or longer-duration trials (≥256 output tokens).

**Thermal trajectory characterized.** We ran a dedicated thermal drift experiment: 5 baseline trials of Q4_K_M at ctx=256 with a short prompt and 64 output tokens (fresh-context, no cooldown), followed by 50 consecutive back-to-back trials (no inter-trial pause), then 5 recovery trials after a 140-second cooldown. Baseline fresh-context throughput was $8.33 \pm 0.58$ tok/s. Note: this baseline is higher than the canonical filled-context measurement (4.78 tok/s, Table II) because with a short input prompt the KV cache grows only from ∼20 to ∼84 tokens during decode, whereas filled-context attention starts from 192 tokens and is proportionally more expensive. Under sustained no-cooldown load, TPS dropped to a steady-state plateau of 4.72–4.96 tok/s within the first 10 trials (elapsed <65 seconds) and remained stable for the next 900 seconds ($\sigma = 0.07$). After the 140-second recovery, TPS partially restored to $7.04 \pm 0.30$ tok/s. This confirms two points: (1) the Tensor G1 throttles under sustained CPU load within ≈60 seconds, reducing fresh-context throughput $\approx 40\%$; (2) steady-state throughput is *stable*—the ±0.07 tok/s noise floor after throttling onset indicates thermal throttling is consistent, not drifting. All primary benchmarks in this paper use a 120-second inter-experiment cooldown, which is sufficient to ensure thermally-stable (not cold-start) conditions. The relative variant ordering reported in Table II is unaffected by thermal state, as all variants were measured under the same cooldown protocol.

### B. Future Work

**Imatrix calibration for mid-range variants.** Importance-matrix-calibrated re-quantization via `llama-imatrix` + `llama-quantize` is tractable (≈45 min for 200 calibration chunks on CPU) and can improve PPL by 2–6% at 4–5 bits. We ran two independent BoolQ evaluations (n=100 each) for Q2_K-imatrix and Q3_K_M-imatrix on the Pixel 6a: Q2_K degraded −4 to −5 pp (65% vs 69% and 64% vs 69% respectively), Q3_K_M degraded −7 to −8 pp (62% vs 69% and 61% vs 69%). A TruthfulQA evaluation shows no degradation (±1 pp: Q2_K 51% vs 50%, Q3_K_M 68% vs 68%). While the BoolQ decrements do not individually reach statistical significance (Wilson CI ≈ ±9.6 pp at n=100), the consistent direction across two independent runs suggests a real negative effect in the −4 to −8 pp range for sub-4-bpw variants. This is consistent with the literature finding that calibration benefit diminishes sharply below 3 bpw and that the Q2_K HellaSwag collapse is a regime failure rather than a weight-precision problem addressable by imatrix. The most tractable target is Q4_K_S (≥4 bpw), where imatrix shows neutral-to-positive effects and may further widen the accuracy lead over Q4_K_M.

**Cross-platform comparison.** Repeating this benchmark on Snapdragon 8 Gen 3, Apple A-series, and MediaTek Dimensity platforms would establish whether the non-monotonic throughput finding is universal across ARM GGUF inference or specific to the Tensor G1's memory subsystem.

**GPU and NPU acceleration.** The Tensor G1 includes an Arm Mali-G78 GPU and a Google TPU block. Integrating these via OpenCL or delegating to TFLite's NNAPI backend could dramatically change the efficiency–quality Pareto frontier.

**Context lengths beyond 2048.** Q2_K's cliff begins at ctx=512 and reaches $-48\%$ by ctx=2048. Whether Q4_K_M and Q8_0 (both stable to ctx=2048 in this study) remain stable at 4096 and 8192 tokens—where KV-cache growth represents a larger fraction of per-step DRAM traffic—is

an open question predictable by $L2/1024$ if per-thread L2 allocation is known for those systems.

**Qwen 2.5 1.5B cliff characterization.** The empirical formula $\text{ctx}_{\text{cliff}} = L2/1024$ predicts ctx=512 for Qwen 2.5 1.5B on ARM (same L2=512 KB), which matches the observed Qwen cliff onset in our n=5 sweep. Notably, despite Qwen's different KV head count (2 KV heads vs. Llama's 8), the same formula applies, suggesting the 1024 bytes/token empirical constant is more robust to model architecture than the raw per-layer K+V formula. A dedicated sequential sweep at $n \geq 10$ trials per context point would further strengthen the model-independence claim.

**Alternative frameworks.** MLC-LLM [7] is not a viable apples-to-apples comparator for this study: its Android runtime is GPU-only (OpenCL/Vulkan) with no ARM NEON CPU path, and it has no Q2, Q5, Q6, or Q8 format equivalents. The closest viable candidates to isolate whether the Q6_K anomaly is llama.cpp-specific are MNN-LLM (explicit ARM CPU NEON/i8mm path) and ExecuTorch [11] with XNNPACK backend, which would test the same SIMD dequantization question under a different kernel implementation.

**KV-cache quantization eliminates the Q2_K cliff but reduces peak throughput.** We evaluated `-ctk q8_0 -ctv q8_0` (8-bit KV cache, halving the fp16 footprint) on Q2_K, Q3_K_M, and Q4_K_M using the filled-context methodology. Results are striking: Q2_K's $-51\%$ cliff (7.50 to 3.69 tok/s, 256→2048) is **eliminated** by KV Q8_0—throughput drops only $-2.6\%$ (4.04 to 3.93 tok/s), flat across all contexts. However, KV Q8_0 reduces Q2_K peak throughput by $-\mathbf{46}\%$ at ctx=256 (7.50 → 4.04 tok/s), because the attention kernel must now dequantize Q8_0 KV entries on every decode step—adding fixed overhead regardless of context length. The crossover where KV Q8_0 becomes beneficial for Q2_K is around ctx=1400 tokens (3.75 tok/s default vs. 4.00 tok/s with KV Q8_0). For Q3_K_M ($-11\%$ cliff) and Q4_K_M ($-12\%$ cliff), KV Q8_0 roughly halves the degradation ($-5\%$) with only a $\sim$1% throughput penalty at ctx=256—a favorable trade for applications targeting ctx$\geq$1024. **Recommendation**: enable KV Q8_0 when ctx$\geq$1400 with Q2_K, or when using Q3_K_M/Q4_K_M at ctx$\geq$1024 for modest but consistent improvement.

**Speculative decoding with a draft model.** Meta's Llama 3.2 1B Instruct provides a natural draft model for speculative decoding [15] with the Llama 3.2 3B verifier. Combined GGUF size (Q4_K_M 3B + 1B) is $\approx$2.7 GB, fitting within the Tensor G1's 6 GB budget. At the memory-bandwidth-saturated decode regime observed here, CPU speculative decoding may yield 1.2–1.8$\times$ speedup with a suitable calibration corpus for the draft model.

**P-core CPU affinity.** llama.cpp supports `--cpu-mask` to pin inference threads to specific cores. On the Tensor G1, the correct performance-core mask is `0xF0` (cores 4–7: 2$\times$Cortex-X1 at 2.8 GHz + 2$\times$Cortex-A76 at 2.25 GHz); mask `0x0F` targets the little cores (cores 0–3, Cortex-A55 at 1.8 GHz) and reduces throughput by $-25\%$ for Q2_K. Measuring `0xF0` vs. the default scheduler is an immediately tractable experiment; community data on similar SoCs suggests 10–25% improvement.

**Hardware performance counter validation.** The mechanistic analysis in this paper is based on static instruction-count analysis of llama.cpp NEON/AVX2 source code. Direct validation via ARM Performance Monitor Unit (PMU) counters—specifically L2 cache miss rate, NEON utilization, and stall cycles per token—would confirm whether the Q6_K throughput penalty is dispatch-bound (many short vector operations) or bandwidth-bound (frequent L2 misses from larger weight tables). Android's `simpleperf` tool can expose ARM PMU counters on non-rooted devices for profiling on the Pixel 6a.

**Broader quality evaluation.** All six benchmarks in this study (BoolQ, ARC-Easy, ARC-Challenge, HellaSwag, MMLU, TruthfulQA) are classification or multiple-choice tasks; the model selects among a fixed answer set. Extending evaluation to generative tasks—GSM8K (grade-school math, chain-of-thought reasoning) and HumanEval (Python code generation and execution)—would test whether the Q2_K HellaSwag collapse (near-random on MCQ) extends to open-ended generation, where degenerate output modes may be harder to detect. Based on the observed Q2_K TruthfulQA failure (50% on yes/no questions = chance) and HellaSwag collapse (19%, below chance), we hypothesize that Q2_K will show severe degradation on GSM8K and HumanEval as well [17]. Preliminary on-device evaluation was attempted but revealed a binary interaction issue between `llama-completion`'s interactive mode and file-based prompt input; results from this attempt are excluded. Fixing the evaluation pipeline and running 50 GSM8K + 50 HumanEval questions per variant is the most impactful remaining quality experiment.

## XI. Conclusion

On CPU endpoints, the standard GPU intuition—higher bit-width yields higher throughput—is wrong across every ISA we tested. We identified three non-obvious phenomena, explained their root causes, and derived a predictive model validated across 1,200+ measurements on ARM, x86, and GPU.

**Non-monotonic throughput (CPU-general).** Q2_K is fastest on both ARM (7.49 tok/s) and x86 (14.1 tok/s); Q6_K is slowest (3.53 and 6.8 tok/s)—a 2.1$\times$ intra-format spread in the wrong direction, reproduced on Qwen 2.5 1.5B. The root cause is SIMD dequantization overhead: Q6_K's split-bit layout requires $\approx$26 NEON operations per block versus $\approx$10 for Q2_K.

**CPU/GPU divide.** Apple M4 Metal reverses the ordering entirely (Q4_K_S fastest, Q8_0 slowest), confirming the non-monotonicity is a CPU property. M4 CPU-only inference outperforms Metal for Q8_0 (1.97$\times$) and Q6_K, because Metal's 4-bit dispatch paths eliminate the dequantization bottleneck that defines CPU throughput ordering.

**KV-cache cliff with predictive formula.** Q2_K throughput collapses $-48\%$ on ARM (ctx=512) and $-50\%$ on x86 (ctx=1300–1400). The threshold is predicted within 5% on

both platforms by $L2/1024$—the context at which the per-layer combined K+V working set overflows L2. Q3_K_M is immune because its higher FFN compute cost masks the proportional attention overhead.

**Practical guidance.** Q6_K is strictly dominated on CPU: slower than all 4-bit variants and no accuracy advantage. Q4_K_S gives the best quality (74% BoolQ); Q4_K_M gives the best throughput-stability balance. Q2_K is viable for throughput-limited deployment but fails structured reasoning tasks (19% HellaSwag). KV-cache Q8_0 quantization eliminates the Q2_K cliff at a $-46\%$ short-context cost.

## References


[1] Meta AI, "Llama 3.2: Lightweight and Vision Language Models," [Blog post], Meta AI Blog, September 2024. [Online]. Available: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[2] G. Gerganov et al., "llama.cpp: Efficient LLM inference in C/C++," GitHub Repository, commit `b1-1a29907`, 2023–2025. [Online]. Available: https://github.com/ggerganov/llama.cpp

[3] G. Gerganov, "GGUF: A File Format for Language Model Weights," GitHub, llama.cpp project, 2023. [Online]. Available: https://github.com/ggerganov/ggml/blob/master/docs/gguf.md [Accessed: Apr. 10, 2026]

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *Proc. ICLR*, 2023.

[5] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in *Proc. MLSys*, 2024.

[6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proc. ICML*, 2023.

[7] MLC-AI Team, "MLC-LLM: Universal LLM Deployment Engine with ML Compilation," GitHub Repository, v0.1.0, 2024. [Online]. Available: https://github.com/mlc-ai/mlc-llm

[8] Qualcomm Technologies, Inc., "On-Device AI: Large Language Models on Snapdragon," Qualcomm AI Research Technical Report, 2024. [Online]. Available: https://www.qualcomm.com/research/artificial-intelligence/ai-research [Accessed: Apr. 10, 2026]

[9] R. Yi et al., "EdgeMoE: Fast On-Device Inference of MoE-Based Large Language Models," arXiv preprint arXiv:2308.14352, 2023.

[10] Y. Song et al., "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," in *Proc. SOSP*, 2024.

[11] Meta AI, "ExecuTorch: On-Device AI across Mobile, Embedded, and Edge for PyTorch," GitHub Repository, 2024. [Online]. Available: https://github.com/pytorch/executorch [Accessed: Apr. 10, 2026]

[12] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," in *Proc. EMNLP*, 2023.

[13] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions," in *Proc. NAACL-HLT*, 2019.

[14] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," arXiv preprint arXiv:1803.05457, 2018.

[15] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proc. ICML*, 2023. arXiv:2211.17192.

[16] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization," in *Proc. NeurIPS*, 2024. arXiv:2401.18079.

[17] S. Li, X. Ning, L. Wang, T. Liu, X. Shi, S. Yan, G. Dai, H. Yang, and Y. Wang, "Evaluating Quantized Large Language Models," in *Proc. ICML*, 2024. arXiv:2402.18158.

[18] S. Laskaridis, A. Antonini, A. Katsianos, N. D. Lane, and S. I. Venieris, "MELTing Point: Mobile Evaluation of Language Transformers," in *Proc. MobiCom*, 2024.

[19] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefler, and D. Alistarh, "SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression," in *Proc. ICLR*, 2024.

[20] T. Dettmers and L. Zettlemoyer, "The Case for 4-bit Precision: k-bit Inference Scaling Laws," in *Proc. ICML*, 2023.

[21] C. Liu, T. Shi, M. Xia, K. Shao, J. Chen, and S. Luo, "MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases," arXiv preprint arXiv:2402.14905, 2024.

[22] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently Scaling Transformer Inference," in *Proc. MLSys*, 2023.

[23] S. Biderman et al., "Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling," in *Proc. ICML*, 2023.

[24] E. Georganas, S. Avancha, K. Banerjee, D. Kalamkar, G. Henry, H. Pabst, and A. Heinecke, "Anatomy of High-Performance Deep Learning Convolutions on SIMD Architectures," in *Proc. SC*, 2018.

[25] R. Y. Aminabadi et al., "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale," in *Proc. SC*, 2022.

[26] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Carpenter, S. Ghodrati, V. Bhatt, A. Mishra, M. Rastegari, and M. Farajtabar, "LLM in a Flash: Efficient Large Language Model Inference with Limited Memory," in *Proc. ACL*, 2024.

[27] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," in *Proc. NeurIPS*, 2022.

[28] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, "QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks," in *Proc. ICML*, 2024. arXiv:2402.04396.

[29] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Proc. NeurIPS*, 2022.

[30] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in *Proc. ICLR*, 2024.

[31] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. SOSP*, 2023.

[32] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[33] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a Machine Really Finish Your Sentence?" in *Proc. ACL*, 2019.

[34] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring Massive Multitask Language Understanding," in *Proc. ICLR*, 2021.

[35] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring How Models Mimic Human Falsehoods," in *Proc. ACL*, 2022.

[36] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer Sentinel Mixture Models," in *Proc. ICLR*, 2017.

[37] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. Jordan, J. E. Gonzalez, and I. Stoica, "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference," arXiv preprint arXiv:2403.04132, 2024.

[38] J. Xiao, Q. Huang, X. Chen, and C. Tian, "Understanding Large Language Models in Your Pockets: Performance Study on COTS Mobile Devices," arXiv preprint arXiv:2410.03613, 2024.

[39] S. Na, G. Jeong, B. H. Ahn, J. Young, T. Krishna, and H. Kim, "Understanding Performance Implications of LLM Inference on CPUs," in *Proc. IEEE IISWC*, 2024.

[40] H. Zhang and J. Huang, "Challenging GPU Dominance: When CPUs Outperform for On-Device LLM Inference," arXiv preprint arXiv:2505.06461, 2025.

[41] T. Nguyen and T. Nguyen, "An Evaluation of LLMs Inference on Popular Single-board Computers," arXiv preprint arXiv:2511.07425, 2025.

[42] P. Tummalapalli, S. Arayakandy, R. Pal, and K. Kundan, "LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load," arXiv preprint arXiv:2603.23640, 2026.

[43] D. Gope, D. Mansell, D. Loh, and I. Bratt, "Highly Optimized Kernels and Fine-Grained Codebooks for LLM Inference on Arm CPUs," arXiv preprint arXiv:2501.00032, 2025.

[44] H. Shen, H. Chang, B. Dong, Y. Luo, and H. Meng, "Efficient LLM Inference on CPUs," in *NeurIPS 2023 Workshop on Efficient Natural Language and Speech Processing*, arXiv:2311.00502, 2023.
[45] P. He, S. Zhou, W. Huang et al., "Inference Performance Optimization for Large Language Models on CPUs," in *ICML 2024 Workshop on Foundation Models in the Wild*, arXiv preprint arXiv:2407.07304, 2024.
[46] T. Zhang, J. W. Yi, B. Yao, Z. Xu, and A. Shrivastava, "NoMAD-Attention: Efficient LLM Inference on CPUs Through Multiply-add-free Attention," in *Proc. NeurIPS*, 2024.
[47] E. Frantar and D. Alistarh, "Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning," in *Proc. NeurIPS*, 2022.
[48] E. Frantar and D. Alistarh, "SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot," in *Proc. ICML*, 2023.
[49] A. Dutta, S. Krishnan, N. Kwatra, and R. Ramjee, "Accuracy is Not All You Need," in *Proc. NeurIPS*, 2024.
[50] S. Wang, G. Ananthanarayanan, Y. Zeng, N. Goel, A. Pathania, and T. Mitra, "High-Throughput CNN Inference on Embedded ARM big.LITTLE Multi-Core Processors," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2254–2267, 2020.
[51] L. Chen, D. Feng, E. Feng et al., "Characterizing Mobile SoC for Accelerating Heterogeneous LLM Inference," arXiv preprint arXiv:2501.14794, 2025.
[52] T. Benoit-Cattin, D. Velasco-Montero, and J. Fernández-Berni, "Impact of Thermal Throttling on Long-Term Visual Inference in a CPU-based Edge Device," *Electronics*, vol. 9, no. 12, p. 2106, 2020.
[53] Qwen Team, "Qwen2.5 Technical Report," arXiv preprint arXiv:2412.15115, 2024.
[54] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li et al., "A Survey on Efficient Inference for Large Language Models," arXiv preprint arXiv:2404.14294, 2024.

## APPENDIX

### A. Build Configuration

The llama.cpp library was compiled from commit `b1-1a29907` using the Android NDK version 29.0.14206865. Compilation flags:

```
-O3 -march=armv8.2-a+dotprod+fp16
-DGGML_USE_CPU_HBM=OFF
-DLLAMA_BUILD_TESTS=OFF
```

The resulting `llama-cli` binary was pushed to `/data/local/tmp/` on the device and executed via ADB shell. The Android app was compiled with Android NDK 29.0.14206865, targeting arm64-v8a, using CMake 4.2.3 and Gradle 8.x. All source code, benchmark scripts, and raw results are available at:

https://github.com/krisdcosta/291_EAI

Artifact commit: `0618b5d` (2026-04-10)

The repository includes all `.jsonl` trial data under `results/`, benchmark automation scripts under `scripts/bench/` (26 scripts), figure generators under `scripts/figures/`, and the quality evaluation pipeline under `scripts/eval/`. The PMU validation script referenced in §VII-A is at `scripts/bench/pixel_neon_perf.sh`. The verified ground-truth metrics table is at `VERIFIED_METRICS_MASTER_TABLE.md`.

### B. Model Artifacts

All GGUF models are Llama 3.2 3B Instruct K-quant variants sourced from Hugging Face (`bartowski/Llama-3.2-3B-Instruct-GGUF`, revision `main`). The seven variants evaluated, with file sizes:

| Variant | File size | Filename |
|---|---|---|
| Q2_K | 1.28 GB | `Llama-3.2-3B-Instruct-Q2_K.gguf` |
| Q3_K_M | 1.68 GB | `Llama-3.2-3B-Instruct-Q3_K_M.gguf` |
| Q4_K_S | 1.93 GB | `Llama-3.2-3B-Instruct-Q4_K_S.gguf` |
| Q4_K_M | 2.02 GB | `Llama-3.2-3B-Instruct-Q4_K_M.gguf` |
| Q5_K_M | 2.37 GB | `Llama-3.2-3B-Instruct-Q5_K_M.gguf` |
| Q6_K | 2.76 GB | `Llama-3.2-3B-Instruct-Q6_K.gguf` |
| Q8_0 | 3.42 GB | `Llama-3.2-3B-Instruct-Q8_0.gguf` |

SHA-256 checksums for all seven GGUF files are provided in `results/model_checksums.sha256` in the artifact repository. The Qwen 2.5 1.5B Instruct cross-validation model (`Qwen2.5-1.5B-Instruct-Q*.gguf`) is sourced from `Qwen/Qwen2.5-1.5B-Instruct-GGUF` on Hugging Face.

### C. Key Benchmark Scripts

nosep

- `pixel_llama_tps.sh` — ARM TPS sweep (ctx=256–2048, n=10)
- `pixel_llama_cliff_filled.sh` — ARM cliff sweep, filled-context (n=10)
- `pixel_kvcache_quant.sh` — KV Q8_0 mitigation experiment
- `m4_llama_tps.sh` — Mac M4 CPU/Metal TPS (`llama-bench`)
- `m4_llama_cliff.sh` — Mac M4 Metal cliff sweep
- `pixel_quality.sh` — On-device quality evaluation (6 benchmarks)
- `pixel_neon_perf.sh` — ARM PMU counter validation (simpleperf)
- `gen_cache_hierarchy.py` — Figure: memory hierarchy diagram
- `gen_cross_platform_rank.py` — Figure: cross-platform bump chart
- `gen_deployment_flowchart.py` — Figure: deployment decision tree

All scripts are in `scripts/bench/` (bench) or `scripts/figures/` (figure generators).

### D. Invocation Template

Each trial was executed with the following command template, where `<prompt>` is padded to exactly `ctx - 64` tokens to saturate the KV cache (filled-context methodology):

```
./llama-cli \
  -m <model>.gguf \
  -p "<prompt>" \
  -n 64 \
  -c <ctx> \
  --no-mmap \
  -ngl 0 \
  -t <threads> \
  --log-format json \
  --seed <trial_seed>
```

### E. Statistical Methodology

Means and standard deviations in Table II are computed across $n$=10 trials per (variant, context) cell using the filled-context protocol (prompt $= N - 64$ tokens, 128 output tokens). Trial 1 is excluded as warmup; trials 2–11 are used. Outlier rejection was not applied; all completed trials within the 600-second timeout are included in analysis.

### F. BoolQ (100 questions)

BoolQ questions are drawn from the SuperGLUE validation split [13]. Each example consists of a passage (avg. $\sim$100 words) and a yes/no question. The prompt template is:

```
Passage: {passage}
Question: {question}
Answer with only yes or no:
```

Responses are scored by extracting the first occurrence of the words *yes* or *no* (case-insensitive) from the model output, within 400 characters following the instruction marker.

### G. ARC-Easy (100 questions)

ARC-Easy questions are drawn from the AI2 ARC Easy test split [14]. Each example presents a 4-choice multiple-choice question. The prompt template is:

```
Question: {question}
Choices: A) {A} B) {B} C) {C} D) {D}
Answer with only A, B, C, or D:
```

Scoring uses exact-match on the predicted letter (A/B/C/D). The `model_output` field in stored JSON records the extracted letter only (e.g., "A"), not the raw llama-completion output, ensuring stored results are directly verifiable and reproducible. Verified accuracy across all seven variants: Q2_K 76%, Q3_K_M 78%, Q4_K_S 81%, Q4_K_M 82%, Q5_K_M 81%, Q6_K 79%, Q8_0 80%.